

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

CSC 413 — Past Questions & Materials
by Renance

Course: CSC 413
Institution: Federal University of Petroleum Resources, Effurun
Papers in this pack: 3
Levels: 400

This study pack was compiled and restructured by Renance for CSC 413 students at Federal University of Petroleum Resources, Effurun. It brings together 3 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 3 — CSC 413 POSSIBLE QUESTION 2025

Level 400 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.

Question 1 (Compulsory)

Programming Paradigms and Language Design

In the context of programming languages at FUPRE:

- Define what is meant by a **Programming Paradigm** and provide one example paradigm. *(2 marks)*
- Differentiate between **Procedural/Imperative** and **Object-Oriented** paradigms, providing one language example for each. *(5 marks)*
- Explain the concept of **Portability** as an objective of language design and why it is important. *(5 marks)*
- Describe the trade-off between **Reliability vs. Cost of Execution** in language design, using an example from the material. *(5 marks)*
- For a software project requiring modular and reusable components, recommend a programming paradigm (e.g., Object-Oriented, Functional) and justify your choice. *(5 marks)*

Total: 22 marks

Question 2 (Compulsory)

Structures of Programming Languages and BNF Notation

- Define **Lexical Structure** in the context of programming languages. *(2 marks)*
- Explain the difference between **Syntactic Structure** and **Semantic Structure**, providing one example of a statement for each. *(4 marks)*
- Describe the role of **Comments** in the lexical structure and their importance in programming. *(4 marks)*
- Using the BNF notation provided, analyze the sentence "The high table makes a good horse" and determine if it is syntactically correct, justifying your answer. *(6 marks)*
- Given the BNF definition for a programming language, analyze the statement $\text{sum1} = \text{sum1} + (\text{a1} + \text{a2}) * (3\text{b} * 4 * \text{b})$ and determine if it is syntactically correct, explaining your reasoning. *(6 marks)*

Total: 22 marks

Question 3

Language Evaluation Criteria and Errors

- Define **Readability** as a criterion for evaluating programming languages. *(2 marks)*
- Explain how **Simplicity** and **Orthogonality** affect the readability of a programming language, providing an example for each. *(4 marks)*
- Describe the role of **Exception Handling** in improving the reliability of a programming language. *(4 marks)*
- Differentiate between **Lexical Errors** and **Semantic Errors**, providing one example for each from the material. *(6 marks)*
- For a program that processes financial transactions, suggest one language evaluation criterion (e.g., Reliability, Writeability) that should be prioritized and justify your choice with respect to the application's requirements. *(6 marks)*

Total: 22 marks



Question 4

Identifiers, Declarations, and Type Checking

- Define **Static Scoping** in the context of programming languages. (2 marks)
- Explain the difference between **Static Type Checking** and **Dynamic Type Checking**, providing one example of a language for each. (4 marks)
- Describe one design issue for **Names** in programming languages, focusing on case sensitivity, and how it varies across languages. (4 marks)
- Using the Pascal program example (Sample2), explain how the variable x in procedure pp is bound under static scoping, and how it would differ under dynamic scoping. (6 marks)
- For a real-time system requiring strict type safety, recommend whether a strongly typed or weakly typed language should be used, and justify your choice with an example from the material. (6 marks)

Total: 22 marks

Question 5

Data Types and Data Structures

- Define **Primitive Data Type** in the context of programming languages. (2 marks)
- Explain the difference between **Scalar Data Types** and **Composite Data Types**, providing one example for each. (4 marks)
- Describe one design issue for **Arrays**, focusing on subscript binding, and how it varies across languages like C++ and Fortran. (4 marks)
- Compare **Arrays** and **Linked Lists** as linear data structures, highlighting one advantage and one disadvantage of linked lists over arrays. (6 marks)
- For a program that needs to store and manipulate a large dataset of student records, suggest a suitable data type structure (e.g., Array, Linked List, Record) and justify your choice based on the program's requirements. (6 marks)

Paper 2 of 3 — CSC 413 note 2025

Level 400 • First Semester • Handout

CSC 413

WHAT IS A PROGRAM?

What is a programming language? A programming language is a set of instructions that provide a way for instructing computer on what operation to perform. It is the key of communicating with the compiler and it has defined characteristics, though it has limited vocabulary, each words in it has precise meaning. A programming language can be used in a step by step fashion to solve complex problems despite its limitations. Many programming language has been written. Each is suited to solving a particular kind of problem.

What do all these programs have in common? (Exam question)

Program written in a programming language must be converted to machine language (0s and 1s) because computer does not understand any language apart from the machine language.

What do you understand by organization of Programming language?

1. It includes programming paradigms that defines how computation is expressed.
2. The main features of Programming language and their impact in the performance of programs written in the languages
3. Development of Programming language
4. Lexical, syntactic and semantics

Programming paradigm: which also implies development in programming methodology. The phase programming paradigm refers to a way of conceptualizing and structuring the task of a computer performance.

Programmers approach problems in diverse ways: One can focus on steps for computation, another can focus on data for computation.

A programmer might use techniques from multiple paradigms.

Some programming languages support single paradigm while some support multiple paradigm.

Paradigm Languages Description

Procedural /Imperative Basic, Pascal, Fortran, Cobol, This is designed around Von Algol, C++ Neumann architecture.

Computation is done via statement/ instruction that change a program state.

Central features include variables, assignment statements, iteration, sequence of commands, etc.

Object Oriented Small talk, java, C++ Formulates programs as a series of objects and methods

that interact to perform a specific task

Declarative/Logic Prolog It focuses on the use of fact and rules to describe a problem. It is a rule based programming language and computation are done through inference process.

Functional LISP, ML Scheme It emphasize the evaluation of expression called functions.

Functions are called rather than execution of command.

Computation are made by applying function to parameter

Event Driven Visual Basic, C#, dBase It focus on selecting user interface element and defining event handling routine that are triggered by various mouse/keyboard activity

LANGUAGE DESIGN

The primary purpose of organizing a programming language is to support the construction of a reliable software. However a language should also be user friendly so it can be easy to write, design,

test, run, document and modify programs written in that language. To understand how these objectives maybe actualized, the issue of language design can be better examined from language evaluation criteria.

Objectives of Language design

These include the following:

1. Ease of use: Note: The expression should be made familiar to the programmer.
2. High level design: It allows complex algorithm to be expressed in compact therefore simplifying it
3. Transferability: it allows a program written in high level language to be run on two or more dissimilar or different computers. Also known as Portability.
4. Ease of Debugging: for example, languages such as Programming language|1 & basic incorporates special component debugger to find error.
5. Ease of documentation: a programming language is successful in communicating to other users as well as to the machine. Problem of maintenance can be reduced by effective & accurate documentation. A programming language can have a considerable effect on documentation, documentation is key. Documentation is a style of programming.

Trade-offs in language design

1. Reliability vs Cost of Execution: example:- java demands that all references to array elements be checked for proper indexing which may result to increased execution cost
2. Readability vs Writeability: Some language provides many powerful operations to be written in compact way but at the expense of poor readability. Note: $K++ :- K = K + 1$
3. Readability vs Reliability: for example: - pointers in c++ are powerful tools and very flexible but are not reliable.

CSC 413 -----

STRUCTURES OF PROGRAMMING LANGUAGES

They are:

- Lexical Structure
- Syntactic Structure
- Contextual Structure
- Semantic Structure

Lexical Structure: This defines the vocabulary of a language. Its units are considered building blocks of a Programming language. Lexical structure of all programming languages are similar, they include

the following kinds:

1. Identifiers
2. Key words
3. Operators
4. Separators
5. Literals
6. Comments

Identifiers: These are names used to represent objects like variables, labels, procedures and functions. Most programming languages require that identifiers begin with an alphabetical letter and can be optionally followed by letters, digits and some special characters.

Key Words: These are names reserved by a programming languages and used to form the syntactic structure of the language.

Operators: These are symbols used to represent the operators, all general purpose language should provide certain minimum operators such as:

1. Relational Operators
2. Logical Operators

Separators: These are symbols used to separate lexical or syntactic use of the language, they include:

space, colon, semi-colon, comma, tab, parenthesis.

Literals: These are values that can be assigned to variables of different types. E.g. integer type literals

are integers, character type literals are any character from the character set of the language.

String

type literals are any string of characters. A string is a sequence or set of characters.

Comments: Any explanatory text embedded in a program. They are used for documentation and clarity in a programming language. It starts with a specific keyword or separator.

Syntactic structure of programming language.

This structure defines the grammar of forming sentences or statements using the lexical units. A high

level programming language normally offers the following kinds of statement.

1. Assignment statement
2. Conditional statement
3. Loop statement

Assignment statements assigns a literal value or an expression to a variable.

Conditional statements: test a condition and branches to a certain state based on a test result which

comes out as true or false, this includes: if, else, else if, switch.

Loop statements: Tests a condition and enters the body of the loop or exits the loop based on the test results. Typical loop statements are: for loops and while loops.

Contextual Structure: this is also referred to as "Static Semantic". It defines the program semantics before dynamic execution. It includes variable declaration, initialization and type checking. Some high level language requires that all variables are initialized when they are declared as the contextual

layer. While some languages do not require that their variables are initialized when they are declared. E.g. `int x = 6. Int y, z;`

This implies that initialization can be done either by the contextual layer or semantic layer.

Contextual structure begins to deal with the meaning of a program. For example, a statement lexically correct might not be contextually correct:

`Int k = 15;`

`Int r = "welcome";`

`Int m= k + r;`

This statement above is logically but not contextually correct. This is because it does not make sense

to add an integer literal to a string literal.

Semantic structure

This structure describes the meaning of a program or what the program does during execution. In most languages, this is no formal definition of semantic structure, informal definition is normally used to explain what each program does. The semantic structure of functional and logical programming languages are normally defined based on the mathematical and logical foundation on which the languages are based.

Error types of different levels

Programming errors can occur at all levels of program and we refer to those errors as lexical, semantic and syntactic errors.

Lexical Errors: all errors at the lexical level can be detected by the compiler. E.g.

```
Int if = 0;
```

```
Double xx? = 0;
```

This declaration can cause error in c++ for many reasons.

Syntactic errors: errors at the syntactic level might not be detected by the compiler (not all errors are detected at this level). E.g.

```
Int main () {
```

```
Int x = 0, y = 4;
```

```
Double z = 0;
```

```
If x == 1: y++ // Syntax error: condition must be guided by parenthesis
```

```
Z = x + y // Missing semi - colon.
```

Contextual errors: these errors are complex and compiler implementation may or may not detect the initialization errors depending on whether they actually complete the initialization expression or not.

They include all the errors excluding the logical errors in:

1. Variable declaration
2. Variable initialization
3. Type inconsistency in assignment. For example:

```
Int y = > / (4+3): //Contextual errors that the compiler
```

```
Z = "Welcome"; //type inconsistency may not detect in assignment.
```

Semantic Errors: Errors in the semantic level include all errors in the statement that will be executed

after compilation. The compiler normally does not detect semantic errors.

CSC 413 5|2|2025

LANGUAGE EVALUATION CRITERIA

The LEC is based on certain characteristics as well as the evaluation criteria:

Criteria

Characteristics Readability Writeability Reliability

Simplicity and orthogonality * * *

Control Structure * * *

Data type and structure * * *

System design * *

Support for abstraction * *

Expressibility * *

Type checking *

Exceptional Handling *

Restricted aliason *

1. Readability: Is an important measure in terms of programs and programming languages. One of the most important criteria for judging a programming language has been the ease with which you can read and understand programs. Readability must be considered in the context of the problem domain.

a. Simplicity: Simplicity of a programming language strongly affects its readability. A language that has a large number of basic components is more difficult to learn than one with fewer number of basic components.

i. Another complicating characteristics of a programming language is feature multiplicity that means having more than one way to accomplish a particular operation. e.g. in c++, a programmer can increment an integer variable in 4 different ways: 1. `Sum = Sum + 1`

2. `Sum += 1`

3. `Sum ++`

4. `++Sum`

ii. Another potential problem is operator overloading in which a single operator symbol can have more than one meaning.

A. Orthogonality: orthogonality in a programming language means that the relatively small set of basic structures can be combined in a relatively small number of ways to build the control and data structures of the language. Every possible combination of basic constructs is valid and meaningful.

B. Control Structures: some of the languages of the 50's and 60's have poor readability as a result of limited control statements where indiscriminate use of GoTo statements severely reduces program's readability. The structured programming revelation of the 1970's gave birth to program that you can read from top to bottom and is easier to understand compared to a program that requires the reader to jump from one statement to another.

C. Data types and Structures: The presence of adequate facilities for defining data types and structures in a language is another significant support to readability. for example, suppose a numeric type is used for an indicator flag but because there are no Boolean types in the language, in such a language we must have an assignment statement to assign 1 to a value: `Large = 1;` but if the language has a Boolean type then we can say: `Large = true` instead of `Large = 1;`

D. Syntax concentration: the syntax has a significant effect on the readability of programs.

1. Identifiers forms: restricting variables or identifiers to limited lengths can affect readability.

2. Special words: program appearance and program readability are strongly influenced by the forms of a language's special words. E.g. `begin`, `end`, `for`, etc. especially in the way compound statements or statement groups are formed in control constructs. Several

languages use matching pairs of special words or symbols to form groups. E.g. c++ requires a pair of opening and closing braces.

3. Semantics: designing statements so that their appearance should at least indicate their use or purpose strongly supports readability. Semantics should follow directly from syntax, in some cases the principle is violated by 2 language constructs that are identical in appearance but have different meaning depending probably on context or usage.

2. Writeability: as a criteria is a measure of how easily a programming language can be used to create program for a chosen program domain. Most of the language characteristics that affects readability also affects writeability.

Support for abstraction

Abstraction means the ability to define and use complicated structure or operation in ways that allow many of the details to be ignored or hidden. It is a key concept in contemporary programming language design. The degree of abstraction allowed by a programming language and the naturalness

of its expression are very important to its writeability. Prog. Lang can support 2 distinct classes of abstraction: Process and Data Abstraction.

Process Abstraction: a simple example of a process abstraction is the use of a sub program to implement a task.

Data Abstraction: an example of data abstraction: consider a binary tree that stores integer data in its nodes such a binary tree can be implemented in c++ by using an abstraction of a tree node

in the form of a simple class with 2 pointers and an integer.

Expressibility: expressibility in a language can refer to several different characteristics. E.g. in c++, the notation: `Sum ++` is more convenient and shorter than `Sum = Sum + 1`.

Reliability: A program is said to be reliable if it performs to its specifications under all conditions, most of the language characteristics that affect readability and writeability also affects readability.

Type Checking: This means testing for type errors in a given program either by the compiler or during program execution. Type checking is a vital factor in language reliability because runtime type

checking is expensive. Compile time type checking is more desirable.

Exception Handling: is the ability of a program to intercept runtime error as well as other unusual conditions detected by the program. It leads to corrective measures which affects readability.

Restricted Aliason: is having 2 or more distinct ways of referencing or names for the same memory location.

CSC 413 19/2/2025

ERRORS OCCURED IN INITIALIZATION

Example of semantic error:

```
int main ()
```

```
int x, y, z; //declaration
```

```
y = 3; //execution
```

```
y = 2; //execution
```

```
z = y/(y-x-2); //semantic error when executing
```

Example of contextual error in initialization:

```
int main ()
```

```
int x = 3, y = 5, z = y/(y-x-2); //contextual error in initialization
```

THE BNF (Backus - Naur form) NOTATION

It is a Meta language that can be used to define the lexical and syntactic structure of another language.

We can define a simple English sentence as follows:

```
<Sentence> :: = <Subject> <verb> <object>
```

```
<Subject> :: = <noun> | <article> <noun> | <adjective> <noun> | <article> <adjective> <noun>
```

```
<Adjective> :: = <adjective> | <adjective> <adjective>
```

```
<Object> :: = <subject>
```

```
<Noun> :: = table | horse | computer
```

```
<Article> :: = the | a
```

```
<Adjective>:: = big | fat | good | high
```

```
<Verb> :: = is | makes
```

The symbol (:: =) means the name of the left hand side is defined by expression on the right hand side. The name in between a pair of angle brackets is non-terminal which means that the name needs to be further defined. The vertical bar (|) means an OR relation. The words between the vertical bars are terminal meaning that they don't require further revelation, and they form the vocabulary of the language.

Review the following sentences:

Fast high big computer is good table 1

The high table is a good table 2

A high table makes the high horse 3

The fast big high computer is good 4

Good table is high 5

A table is not a horse 6

Is fast computer good 7

The first sentence is syntactically correct even though it doesn't make sense, the 3 adjectives in the

sentence are correct because from the definition, adjectives are recursive.

The second and third sentences are also syntactically correct according to the definition.

The fourth and fifth are not syntactically correct because a noun is missing in the object of the

sentences

The sixth sentence is incorrect because 'not' is not in the vocabulary.

The last sentence is incorrect because the definition does not allow a sentence to begin with a verb.

BNF OF A PROGRAMMING LANGUAGE

<Letter> :: = a|b|c|d|e|...|Z

<Digits> :: = 0|1|2|3|4|5|6|7|8|9

<Symbol> :: = _|@|.|~|?|#|\$

<Char> :: = <letter> | <digit> | <symbol>

<Operator> :: = +|-|*|/|<|>|=|=|>|=|<|=|and|or|not

<Identifier>:: = <letter>| <identifier> <char>

<Number> :: = <digit>|<number><digit>

<Item> :: = <identifier>|<number>

<Expression> :: = <item> |(<expression>)|<expression><operator><expression>

<Branch> :: = if <expr> then {<block>}| if <expr> then {<block>} else {<block>}

<Switch> :: = Switch <expr> {<sbody>}

<Sbody> :: = <cases>|<cases>; default: <block>

<Cases> :: = case<value>: <block> |<cases>; case<value>: <block>

<Loop> :: = while<expr> do {<block>}

<Assignment> :: = <identifier> = <expr>;

<Statement> :: = <assignment> | <branch> | <loop>

<Block> :: = <statement>|<block>;<statement>

sum = 0; 1

while sum <= 20 do{...} 2

sum1 = sum1 + (a1 + a2) * (3b * 4 * b) 3

if sum1 ==100 then 2sum = sum1 else sum2 + sum1; 4

p4 = ((1a + a2) * (b3 * b4)) / (c7 - c8) 5

_kk = (a1 + a2 - b3 - b4); 6

(a1/a2) = (c3 - c4) 7

According to the BNF definition of a programming language, the following sentences could be analysed as follows:

Statements one and two are syntactically correct.

Statements three and four are not correct because you can't begin an identifier with a number.

Statement 5 is not correct because you can't begin an identifier with a number.

Statement 6 is not correct because identifiers cannot begin with an underscore.

CSC 413 26th February 2024

IDENTIFIERS, DECLARATION AND TYPES

The choice of a language applies to the choice of a set of data types and data structures. Therefore the treatment of data is one of the most important aspects of programming language design.

Names: A name is a string of characters used to identify some entities in a program.

Design issues for names

The following are the primary design issues for names

1. What is the maximum length of a name?
2. Can connector characters be used as a name or in names?
3. Are names case sensitive?
4. Are the special words reserved or key words

Different languages has its unique types on objects that are allowed, the operations that can be applied and the methods of controlling the sequence data objects have a value and a type and may be stored in a variable. Data objects is a region of storage that contains a value or a group of values. Variables have a name and various attributes refer to an area of a memory cell in memory in

a computer, therefore it is necessary to make distinction between:

1. The name of a variable i.e. its identifier
2. The area of memory location i.e. its reference or address
3. The data object stored.

Usually the names of variables are often called identifiers. An identifier is a combination of letters and digits with the first character being a letter. Embedded blanks or spaces are not normally allowed within identifiers. Languages such as C, C++ have found a compromise by using the underscore character. Languages have different lengths for an identifier. E.g. FORTAN 77 restricts the length of 6 characters while languages such as PASCAL, C++, and C allow identifiers of any length.

Lower and upper case letters considerations: in some programming languages, lower and upper case

letters are distinct, such as C++, while in PASCAL and DBASE, case does not matter.

Declarations and Binding: The act of associating attributes with a variable is known as binding, the most important features of a language in determining the treatment of binding is the declaration. Languages with declarations with includes most widely used languages today generally early any binding of many attributes. When an attribute is bound to a variable by means of declaration statement, the association is usually firmly established and may not subsequently be changed. E.g. when the variable K has been declared as an integer, it is usually impossible to treat it as non-integer, however, non-integer values may be assigned to K, but they will be transformed or converted to an appropriate integer values. In languages without declarations, attributes may be bound to a variable by means of an assignment statement. Binding can take place at compile time. In

general, the ability to bind 2 features early may lead to efficient programs while binding that is delayed on the run time may lead to more flexibility.

Program Sample (input, output),

Var x, y: real,

Procedure assignment;

Var y, z : integer;

begin

x := 17.5;

y := 60;

end (assignment);

begin

x := 26.5;

y := x;

end (sample);

Blocks: in the example program, the main program (sample) and the procedure (assignment) are referred to as blocks. Blocks may be nested within other blocks.

Names declared in a block are not visible from outside that block. The integer variables, y, and z are

not visible outside the procedure assignment. They are called local variables. Variables used in one block but declared in an enclosing block are called global or non-local variables.

Languages like FOTRAN, PASCAL, ADA, etc. and the binding of declarations is determined at compile

time and this is referred to as static scope, a binding is static if it occurs before runtime and remains

unchanged throughout program execution. The advantage of static scope is that it allows the checking to be performed by the compiler which are termed strongly typed. In dynamic scope, the binding between the use of identifier and its declaration depends on the order of execution and as a result it is delayed until runtime. A binding is dynamic if it occurs during runtime or can change in the

course of program execution.

Illustration: in PASCAL language.

Program Sample2 (input, output);

Var x: integer,

Procedure pp;

begin

...write (x); ...

end (pp)

Procedure cc;

Var x: real;

begin

..pp; ...

end (cc)

begin

...pp;

...cc;

End (Sample)

Pascal has a static scope rule, and therefore the use of variable *x* in procedure *pp* is bound to the declaration of the integer *x* in the main program (Sample2). In a dynamic scope, the use of *x* in procedure *pp* would bound to the most recent declaration of *x*. hence, when procedure *pp* is called from procedure *cc*, the use of *x* in *pp* would be bound to the declaration of the real variable *x* in procedure *cc*, while when *pp* is called from the main program, the use of *x* would be bound to the declaration of the integer variable *x* of the main

Before a variable can be referenced in a program, it must be bound to a data type. The two important aspects of binding involves:

1. How the type is specified and
2. When the binding takes place

Types can be specified statically, through some ways of implicit or explicit declaration.

An implicit declaration is a means of associating variables with types through default conventions instead of declaration statements.

A = 26.5

The variable *A* is a real variable because it was assigned a real value, if it

An explicit declaration is a statement in a program that lists variable names and declares them to be

a particular type.

E.g. INTEGER *A*, *BB*, *C*

Dynamic type binding: with dynamic type binding, the type is not specified by declaration statement,

instead the variable is bound to a value in the assignment statement.

E.g. Int *X*, *Y*, *Z*;

X = 24;

Y = 66.44;

When the assignment statement is executed, the variable being assigned is bound to the type of the

value, variable, expression on the right hand side of the assignment.

CSC 413 12/3/2025

TYPE CHECKING

This is the activity of ensuring that the operands of an operator are of compatible types. A compatible type is one that is either legal for the operator or is allowed under language rules to be implicitly converted by the compiler to a legal type, this automatic conversion is usually referred to as coercion. A type error is the application of an operator to an operand of an inappropriate type.

Type checking is complicated when languages allows a memory cell to store values of different types

at different times during execution. In this case, type checking if done, must be dynamic, and requires the runtime system to maintain the type of the current value of such memory cells.

Static type checking occurs if all bindings of variables to types are static in a language.

Strongly typed languages: a strongly typed language is one in which each name in a language has a

single type associated with it. And that type is known at compile time. The essence of this definition is that all types are statically bound. We define a programming language to be strongly typed if type

errors are always detected. The importance of strong typing lies in its ability to detect all misuses of variables that results in type errors. A strong type language also allows the detection at runtime or uses of incorrect type values in variables that can store values of more than one type. Therefore a strongly typed language has a stricter typing rules at compile time which means that errors are more

likely to happen at compile time. And most of these rules affect variable assignments, function returning values, procedure arguments and function calling. Dynamic type languages where type checking happens at runtime can also be strongly typed. In dynamically typed languages, values rather than variables have types.

Weakly typed languages: a weakly typed language has looser typing rules and may produce unpredictable or even erroneous results or may perform implicit type conversion at runtime. Weak typing implies that the compiler does not enforce a typing discipline. A weakly typed language is one

where the bits of a value can be reinterpreted as another type or there are too many coercions available or the coercions that exists are too error prone. Such languages provide greater freedom when it comes to variable type. Variables can have many data types assigned to them and type conversions are handled dynamically by the compiler or interpreter. E.g.

In Java

```
String k = "xyz" + 246; // "xyz246"
```

This code is an example of implicit conversion. 246 is implicitly converted to a string before being concatenated with "xyz".

Java "rewrites" the code to be: `String k = "xyz" + new String (246);`

Examples of strongly typed languages:

1. C sharp
2. Java
3. Python
4. RUBY

Examples of weakly typed languages

1. FOTRAN 77
2. C
3. CPP

4. JAVASCRIPT

Structuring Data

2 categories of structuring data.

1. Data types
2. Data Structures

Data Types in Language Design: a data type is the most basic and most common classification of data. It is an attribute of data which tells the compiler how the programmer intends to use the data.

Data type can be grouped into 3 categories:

1. The scalar data type
2. The Composite and
3. The Abstract data type

The Scalar data type is basically the building blocks that has to do with the Boolean, integer, float, char etc. The composite data type has to do with the array, String, struct, etc.

The Abstract data type is defined by its behaviour. The tuple, set, stack, queue, graph, etc.

The Boolean: the Boolean type represent the values: true or false. It has two distinct values, 0 and 1.

The 0 represents false, the 1 represents true.

The Numeric types: numeric is a data type that portrays constants or a number. There are different versions of numeric, it could be an integer or it could be a floating point/scientific, or fixed format.

Composite Types: composite types are derived from more than one primitive type they are: Array, union, set, object, records.

The String type: a string is simply a sequence of character, character is a letter, or digit or symbol, char is a scalar data type.

Abstract Data Types: any type that does not specify an implementation is an abstract data type. E.g.

a stack can be implemented as an array or a linked List. Abstract types can be handled by code that

does not know or care what underlying types are contained in them.

CSC 413 26/3/2025

DATA TYPES

They are the most important aspect of variables, its characteristics are, it allows values together with

a set of operators that can be used to manipulate these values.

Most Common Data types:

1. Numeric types
2. Logical types
3. Character types

Built in data types are readily available to the programmer.

Fortran Pascal C C++

Integer Integer Int/unsigned int Int/short/long/unsigned
int/unsigned short/
unsigned long

Real Real Float Float, double, long
double

Logical Boolean Char/unsigned char
Character Char Char Char / unsigned char
Double precision Double

Primitive type: The data type that are not defined in terms of other data types are called primitive data types, nearly all programming languages possess a set of primitive data types

Numeric data type: Many early programming languages had only numeric primitive types and these types still play a major role in many contemporary programming language supported type.

Integer: The most common primitive data type is an integer, many computers now support several sizes of integer.

Floating point: The floating point data type models real variables but its representatives are only approximations for most real values. Floating point values are represented as functions and exponents with different implementation. This data type is included in most programming languages. Languages that are designed to support scientific programming generally includes two floating point types often called "real and double precision". E.g. in Fortran, the real type is the standard size and the value of the real type is stored in a single memory word, but the double precision type is provided for situations where longer fractional parts are required, double precision variables usually occupy twice as much storage as real variables and provide at least twice the number of bits in storage.

Boolean: Boolean types has only two values and are representing true or false (i.e. 1's or 0's respectively)

Character types: character types are stored in computers as numeric codings. The most commonly used coding is "ASCII" (American standard code for information interchange) which was the values 0

- 127 to code 128 different characters.

Array/Array types: An array is a contiguous memory location provided to store values with a single name.

Design issues in Arrays

The primary design issues specific to arrays are

1. What types are legal for subscripts?
2. When are subscripts range bound
3. When does array allocation take place?
4. How many subscripts are allowed
5. Can arrays be initialized when they have their storage allocation?
6. What kind of slices are allowed if any?

Array and subscripts

Arrays are referenced by subscripts, if all subscripts in a reference are constant, the selection is static, otherwise, it is dynamic – Fortran makes use of parenthesis for array subscript but Pascal, C, C++, uses square brackets “[]”

Subscript Binding & Array Categories

The binding of subscript type to an array variable is usually static but the subscript value ranges are

sometimes dynamically bound, two distinct types involved in an array type are listed below:

1. The element type
2. The type of subscript (The type of subscript is usually a subrange)

In some languages, the lower bound of the subscript range is implicit for example, in c & c++, the lower bound as all subscript range is fixed at 0 (zero).

Number of subscript in Arrays

FORTRAN limited the number of array subscripts to be between 3 – 7 but most other contemporary languages enforces “no limit”.

Array Initialization: some languages provide the means to initialize arrays at the time their storage is

allocated in Fortran 77. For instance, all data storage is statically allocated. For example:

Integer k [3]

Data k [4, 6, 0]

Character string in c and c++ in the form of initialization. This array can be initialized to String constant.

Char name [] = “Welcome”;

Pascal does not allow array initialization in the declaration section of programs

Data Structures

A Data Structure is a collection of a data type value which are stored and organized in such a way that it allows for efficient access and modification. Data structure performs some special operations like insertion, deletion and transversal, they can be grouped into four categories.

1. Linear data structures: we have the arrays and lists
2. Tree Data Structures: we have binary heaps and space partitioning.
3. Hash Data Structures: e.g. distributed hash table hash tree, etc.
4. Graph Data Structure: we have decision, directed and acyclic etc.

Linear data structures: Look back into arrays.

Linked lists: a linked list is different from an array in that the number of elements in the list are not determined by contiguous memory location, instead the memory of the linked list can be sporadically placed in memory due to its design, which is that each element of the list consist of two

parts. “The data” and “The pointer”. Also in a linked list, there is no index access so in order to locate

a specific piece of data, you need to transverse the entire list until you find the data you’re looking

for.

Paper 3 of 3 — CSC 413 handout

Level 400 • First Semester • Handout

Department of Computer Science

Federal University of Petroleum Resources, Effurun,

Delta State

CSC413:

ORGANIZATION OF PROGRAMMING LANGUAGES

[ORHIONKPAIYO B. C.]

CSC413: Organization of Programming Languages

1. COURSE GOAL

This course is intended to acquaint students with in-depth knowledge of some language constructs and the rationale for those constructs in programming languages. The intention is to improve the ability of the students to express themselves programmatically and to increase their ability to learn new programming languages.

2. COURSE OBJECTIVES

At the end of this course, students should be able to:

- i. Enumerate and discuss the domains of programming languages
- ii. List and discuss the attributes of a good programming language
- iii. Explain the factors that influence the design of programming languages
- iv. Compare the different programming paradigms
- v. Discuss the benefits of specific programming paradigm
- vi. Give reasons why there are so many programming languages
- vii. Explain the characteristics of specific programming paradigms
- viii. Discuss the methods for implementing programming languages
- ix. List and explain the attributes of variable
- x. Differentiate between static and dynamic binding
- xi. Illustrate how the attributes of variable are implemented in some programming languages
- xii. Enumerate the design issues of some attributes of variable
- xiii. Differentiate between some specific data types
- xiv. Discuss the design issues of some data types
- xv. Illustrate how data types are implemented in some programming languages
- xvi. Explain the control flow and execution sequence in different programming languages
- xvii. Describe the syntactic and semantic structures of programming languages
- xviii. Describe the phases of program translation from source code to executable code and the files produced by these phases.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 2

CSC413: Organization of Programming Languages

3. COURSE DESCRIPTION

The course discusses the fundamental concepts of programming language particularly the design issues of the various language constructs (such as variables and data types), examining the design choices for these constructs in some of the most common languages and critically comparing design alternatives. It also includes factors that influence the design of programming languages and the implications of the different programming paradigms. The course also introduces the fundamental syntactic and semantic concepts underlying modern programming languages.

4. COURSE SCHEDULE

WEEK TOPICS SUB TOPICS

1 Course Orientation and the Review of course syllabus
syllabus

2 Attributes of a good i. Why study programming languages?

Programming language

ii. Attributes of a good programming
language

iii. Why so many programming
languages?

Domain of Programming i. Scientific
languages

ii. Business

iii. Artificial Intelligence

iv. System Programming

v. Web Software/Scripting languages

3-4 Influences on Design of i. Computer architecture
programming languages

ii. Programming design methodologies

iii. Overview Programming paradigms

a. Structured

b. Imperative

c. Object-Oriented

d. Functional

e. Logic

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 3

CSC413: Organization of Programming Languages

5 Programming language i. Compilation

Implementation methods ii. Pure Interpretation

iii. Hybrid

6-7 The concept of variables in Sextuple attributes and design issues:

programming languages i. Name

ii. Address

iii. Value

iv. Type

v. Lifetime

vi. Scope

8-9 Data types in programming Data types & design issues:

languages i. Primitive data types

ii. Character/String

iii. User defined

iv. Array

v. Records

vi. Type checking

10 Control Structures i. Selection statement

ii. Iterative statement

iii. Unconditional branching

11 - 12 Lexical analysis & parsing i. Describing language Syntax & Semantics

ii. Lexical Analysis & Parsing

13 Test & Revision

5. CONTACTS

6. GRADING

Attendance - 3%

Assignments/Laboratory - 15%

Test - 12%

Exam - 70%

7. POLICIES

(i) Attendance - It is expected that every student will be in class for lectures. Attendance records will be kept and used to award marks for each student. Only students with up to or above 75%

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 4

CSC413: Organization of Programming Languages

attendance will be awarded 3mks. Signing for friends is an act of DISHONESTY and it is HIGHLY PROHIBITED.

(ii) Makeup test - There shall be no makeup test. The date for the test shall be announced to the class at least one week before the test.

(iii) Academic Integrity – Any form of academic dishonesty, such as copying during test /exam or making copies of other

students' work, is prohibited.

(iv). Submission of Assignments/Lab work – Late submission of assignment or lab work will be graded as $\text{Score} - x$, where x is the number of days of late submission.

8. IMPLEMENTATION TOOLS

(i) Java Development Kit (JDK). Download this at:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

(ii) Eclipse Integrated Development Environment (IDE).

Download this at <https://eclipse.org/downloads/>

9. TEXTBOOKS

i. Concepts of Programming Languages, 11th Edition, Robert W. Sebesta, Addison-Wesley, 2016.

ii. Introduction to Automata theory, languages and computations- John E. Hopcroft and Jeffery D. Ullman

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 5

CSC413: Organization of Programming Languages

1.1 Why Study Programming Languages?

A programming language is a vocabulary and set of grammatical rules (syntax) for instructing a computer to perform specific tasks. Programming languages are primarily intended for expressing algorithms in a form that can be executed by a computer.

Some of the reasons for studying programming languages include:

- To improve your ability to develop effective algorithms. In programming language, the complexity of the algorithms that people implement is influenced by the set of constructs available in the programming language
- To allow a better choice of programming language. Many programmers use the language with which they are most familiar, even though poorly suited for their new project. It is ideal to use the most appropriate language.
- To make it easier to learn a new language. In software development, continuous learning is essential. The process of learning a new programming language can be lengthy and difficult, especially for someone who is comfortable with only two or more languages. Once a thorough understanding of the fundamental concepts of languages is acquired, it becomes far easier to see how these concepts are incorporated into the design of the language being learned.
 - o For instance, knowing the concepts of object oriented programming OOP makes learning Java significantly easier

and also, knowing the grammar of one's native language makes it easier to learn another language.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 6

CSC413: Organization of Programming Languages

1.2 Attributes of a Good Programming Language

The attributes have been grouped under three main categories:

1. Attributes Relating to Ease of Using a Language
2. Attributes Relating to Software Engineering
3. Attributes Relating to Performance

I. ATTRIBUTES RELATING TO EASE OF USING A LANGUAGE

Programming languages are used by programmers to write programs. Thus, a good language should make it easy for a programmer to express what needs to be done. Several criteria contribute to making a language easy to use.

□ **WELL-DEFINEDNESS:** By "well-definiteness", we mean that the language's syntax and semantics are free of ambiguity, are internally consistent and complete. Thus the implementer of a well defined language should have, within its definition a complete specification of all the language's expressive forms and their meanings. The programmer, by the same virtue should be able to predict exactly the behavior of each expression before it is actually executed.

Syntax answers the question "What forms does the language allow?" This is important, so that a programmer knows how to construct statements that will be accepted by the compiler. If the syntax definition is ambiguous, then the programmer may have to resort to trial and error. Even worse, different compilers for the same language may differ in their interpretation of an ambiguity, leading to portability problems. There are a number of notation systems which can be used to spell out the syntax of a language. Any one of these systems can be used to spell out syntax unambiguously, though some are more readable for humans than others are.

Semantics answers the question "what does this form mean?" The importance of this for the programmer is obvious. Ambiguity here may again force the programmer to resort to trial and error.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 7

CSC413: Organization of Programming Languages

Example: a classic example of ambiguous semantics is the "dangling else" problem:

should a construct of the form

if (B1) if (B2) S1 else S2

[where B1 and B2 are Boolean expressions and S1 and S2 are statements]

be interpreted as:

```

if (B1)
if (B2)
S1
else -- else goes w/second if
S2
or as
if (B1)
if (B2)
S1
else -- else goes w/first if
S2

```

- Obviously, the problem can always be avoided by using braces (or its equivalent) around the inner if. But this is inconvenient and programmers don't usually do this.

- ALGOL handled this by forbidding the "then" part of an if from being another if.

Should a programmer want a construct like this, he would be FORCED to use
begin ... end

- Some newer languages require an explicit "end if" to terminate an if .. then .. else, also avoiding the problem (e.g. FORTRAN77, Modula-2, Ada)

- Most languages resolve the ambiguity as Java does, by matching the else with the NEAREST if that has no else (the first interpretation above.) But this rule is often not clearly stated in the language manual (though in the case of Java it is!)

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 8

CSC413: Organization of Programming Languages

Further, if the semantics of language construct are ambiguous, and two compilers interpret the ambiguity differently, then when a program is ported from one compiler to another it may no longer run correctly - an even worse problem than that arising from a syntactic ambiguity, since the compiler will give the programmer no indication of the problem.

Example: in a Boolean expression like

```
(i < max) && (x[i] > 0),
```

is the second comparison done if the first fails? This is an important question if the second comparison would cause a run-time error if the first comparison is false.

- Java addresses this question by saying that, in a case like this, the second comparison is NEVER done if the first fails.

- Ada addresses this question by saying that both comparisons are ALWAYS done. It provides a different construct for use when one wants to not avoid doing the second comparison: (i < max) and then (x[i] > 0)

- Some other languages leave this question unresolved, and different compilers may produce different results. (In fact, in some cases the SAME compiler may handle

this differently in different context – this lead to some problems in old Pascal compiler)

□ EXPRESSIVITY: Expressivity means the ability of a language to clearly reflect the meaning intended by the algorithm designer (the programmer). Thus an “expressive” language permits an utterance to be compactly stated, and encourages the use of statement forms associated with structured programming (usually “while “loops and “if – then – else” statements). This means consistency with commonly used notation

This point can be illustrated best by looking at some violations of this criterion.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 9

CSC413: Organization of Programming Languages

1. In writing mathematical expressions, the normal practice is to write the arithmetic operators in infix form - e.g. we write

$x + 1$ (infix)

instead of

$+ x 1$ (prefix)

or $x 1 +$ (postfix)

Most programming languages adhere to this convention, though some don't - notably LISP (prefix) and FORTH (postfix). This makes programs in these languages harder to read and write - though one eventually gets used to it.

2. Again, in writing mathematical expressions, certain conventions are normally understood with regard to operator precedence. For example,

$3 * x + 2$

is normally understood to mean

$(3 * x) + 2$

Most programming languages adhere to conventional rules of operator precedence, but some do not. For example, in APL the unparenthesized expression would be interpreted as

$3 * (x + 2)$

3. In conventional mathematical notation, the $=$ operator means the ASSERTION that two things are equal, as distinct from the assignment operation MAKE two things equal.

a. Thus, Algol (and its descendants through Pascal) used $=$ for comparison and $:=$ for assignment.

b. Some other languages use $=$ for both purposes, leading to possible misreadings. (Examples: BASIC, COBOL).

c. Other languages use $=$ for assignment and something else for comparison, contrary to conventional mathematical usage.

(Examples: FORTRAN, and the C descendants of ALGOL including Java).

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 10

CSC413: Organization of Programming Languages

□ Good facilities for INPUT-OUTPUT.

1. This turns out to be one of the hardest parts of a language to design, because there is an inevitable dependence here on distinctive characteristics of different hardware devices. Facilities for interactive IO to terminals, for example, must be somewhat different from those for reading/writing disk files.
2. One issue that languages handle quite differently is whether the IO facilities of a language should be part of the language definition itself, or provided by library procedures.
 - a. FORTRAN, COBOL, and a number of others take the first approach; with the language including READ and WRITE statements.
 - b. C and its descendants (including Java) and Ada (among others) take the second approach. IO operations are done through calling library procedures. A basic IO library is furnished with the language system, but a programmer is free to extend it to handle special needs.
3. Another issue is facilities for FORMATTING output. COBOL is very strong on this count, and others (like FORTRAN) include good facilities; while other languages are quite weak. (For example, formatted output is very hard to do in Java).

□ UNIFORMITY. That is, similar constructs should have similar meanings.

Again, this can be illustrated by a counter-examples:

1. In the C family of languages (including Java), parameters to functions are normally passed by value. Thus, given the C function definition

```
int f(x)
int x;
x = 2 * x;
return x;
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 11

CSC413: Organization of Programming Languages

and a call to the function

```
int a = 2;
b = f(a);
// a still has the value 2 here
```

The assignment to the formal parameter x in f has no effect on the actual parameter a.

2. But if the parameter is an array, then it is passed by reference instead. Thus, given the very similar function definition:

```
int f(x)
int x[];
x[1] = 2 * x[1];
```

```
return x[1];
and a call to the function
int a[2];
a[0] = 2;
b = f(a);
// The value of a[0] is now 4!
```

The assignment to the formal parameter *x* in *f* WILL ALTER the first element of the actual parameter *a*.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 12

CSC413: Organization of Programming Languages

□ ORTHOGONAL: A similar concept is that a language should be orthogonal.

Orthogonality in a programming language means that a relatively small set of primitive constructs can be combined in a relatively small number of ways to build the control and data structures of the language. An orthogonal language is one which has a limited number of features, each of which can be understood by itself, which can be combined in any way to produce a variety of results. For example, if the language has three primitive data types, an integer, double, and character, and two type operators, array and pointer, then a large number of data structures can be defined

1. This was a major design goal of ALGOL68, but has been less characteristic of most other languages.
2. As a counter-example, consider the matter of types and functions in Pascal.
 - a. In Pascal, a variable can be declared to be of any type, and a variable of a given type can be assigned a value of that type by an assignment statement.
 - b. A function can take parameters of any type, but cannot return values of certain types. (Specifically, it cannot return an array or a record.)
 - c. That is, to use Pascal, one must not only learn about types and functions, but must also learn the rule that functions cannot return certain types. These two otherwise unrelated concepts in the language interact in an unusual way. In a truly orthogonal language, functions would be able to return values of any type.
 - d. The difficulty can be pictured this way:

Use

| Variable | Parameter | Return value

Type Scalar | OK | OK | OK |

-----+-----+-----+-----|

Structured | OK | OK | No! |

(In an orthogonal language, all six squares would be OK)

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 13

CSC413: Organization of Programming Languages

□ PEDAGOGY – A language should be easy to learn. Some languages have better “pedagogy” than others. That is, they are intrinsically easier to teach and to learn, they have better textbooks; they are implemented in a better program development environment, they are widely known and used by the best programmers in an application area.

1. Several of the features we have already considered contribute to pedagogy - e.g. consistency with commonly used notation, uniformity, orthogonality.
2. On the other hand, abundance of features tends to make a language hard to use.
 - a. That is, generality can sometimes conflict with pedagogy.
 - b. However, it is possible to achieve a good balance between both.

One of the reasons for Pascal's popularity as a teaching language is that it is powerful, but relatively small. One can master the entire language in an introductory course sequence.

II. Attributes Relating to Software Engineering

A. Beyond ease of use, it is important that a programming language support the development of CORRECT software, even when writing large systems. The next group of criteria we consider pertain to support for good software engineering.

B. It is important that the language make it difficult to make careless errors that go undetected by the compiler. Horowitz calls this characteristic RELIABILITY.

A program is said to be reliable if it performs to its specifications under all conditions.

1. Many early programming languages - and some recent ones - do not require that a variable be explicitly declared by the programmer.
 - a. For example, in FORTRAN, a variable that is not explicitly declared is implicitly declared the first time it is used, with its type being determined by the first letter of its name. (Names beginning with I..N are integers; all others are real)
 - b. Consider what happens if a programmer makes a typographical error, misspelling the name of a variable. In a language that requires that all variables be

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 14

CSC413: Organization of Programming Languages

declared, this will almost always be caught by the compiler as an "undeclared identifier" (unless the typo happens to come out the same as another variable.) In languages like FORTRAN, though, the compiler will usually not catch such an error.

- c. Of course, it may be argued that the requirement of declaring every variable is an inconvenience for the programmer. But, in this case, the inconvenience of having to track down a subtle bug due to a mistyped variable is even worse, so it's worth it.

2. A reliability feature related to the requirement of declaring variables is type checking.

Type checking is simply testing for type errors in a given program, either by the compiler or during program execution. Type checking is an important factor in language reliability. Because run-time type checking is expensive, compile-time type checking is more desirable. Furthermore, the earlier errors in programs are detected, the less expensive it is to make the required repairs. The design of Java requires checks of the types of nearly all variables and expressions at compile time. This virtually eliminates type errors at run time in Java programs.

a. As you know, languages like Java check that the use of a variable is consistent with its declaration. For example, the following will be detected as erroneous:

```
boolean b;
```

```
System.out.println(b*2);
```

b. Many languages do not do this. For example, consider the following legal FORTRAN program:

```
SUBROUTINE SUB(A)
```

```
INTEGER A(100)
```

```
DO 10 I = 1,100
```

```
100 A(I) = A(I) + 1
```

```
END
```

```
REAL B, C
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 15

CSC413: Organization of Programming Languages

```
CALL SUB(B)
```

The compiler will permit this, and on the first time through the loop the subroutine will treat the bit pattern for the real parameter B as if it were an integer, producing strange results from the addition. Even worse, on subsequent times through the loop it will move past the memory allocated to B. Thus, the operation on A(2) will probably be done on C, and the operation of A(3) .. A(100) on who knows what - perhaps even the code will be damaged!

3. Finally, the commenting conventions of a language are also a factor in reliability.

a. Languages tend to approach commenting in one of three ways.

i. In some languages, comments occupy entire lines unto themselves.

For example, FORTRAN uses a C in column 1 to specify a comment line, and COBOL uses a * in column 7 for the same purpose.

ii. Other languages use pairs of comment delimiters to bracket comments. A comment may start and end anywhere, so you can have comments in the middle of a line or extending over a full line or many lines. For example, Pascal uses (* and *) or { and } this way; C, Java and PROLOG use /* and */, etc.

iii. Finally, some languages use a symbol to start a comment, which may appear anywhere on the line. The comment occupies the rest of that line - i.e. the end of the line closes the comment.

For example, Lisp uses ; this way and Ada uses -- this way:

```
if DISCR < 0.0 then -- roots are complex
```

```
S := sqrt(-DISCR)
```

iv. Also, there are languages that use multiple approaches - e.g. Java supports "remainder of line" comments using //

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 16

CSC413: Organization of Programming Languages

b. What convention is most reliable? Obviously, one will get differences of opinion on this point, but one approach does have the edge.

i. The approach taken by FORTRAN and COBOL tends to discourage commenting, because a comment occupies an entire line. In particular, it is hard to associate a comment about what a variable is used for with its declaration - a good practice.

ii. The approach taken by Java and many other languages suffers from the danger of mistyping the closing comment bracket. For example, almost everyone at some time has added some comments to a working program. If one mistypes the closing bracket (e.g. as */ or the like), then all of the code between that bracket and the end of the NEXT comment is "commented out". If the code happens to still be syntactically correct, a working program may cease to work without warning.

iii. The designers of Ada studied typical errors made by programmers, and concluded that the convention they adopted was the most reliable.

□ MODULARITY Another software engineering consideration is the language's support for modularity. Modularity has two aspects: the language's support for sub-programming and the language's extensibility in the sense of allowing programmer - defined operators and data types. By sub programming, we mean the ability to define independent procedures and functions (subprograms), and communicate via parameters or global variables with the invoking program.

Large software project is typically constructed of modules, each of which interfaces with the rest of the system in certain well-defined ways.

1. A subroutine facility, such as that found in FORTRAN or COBOL, or a block-structured procedure facility, as in ALGOL or Pascal, is one way to address this need; but such facilities are not as good as they might be.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 17

CSC413: Organization of Programming Languages

2. Some languages, such as Modula-2 and Ada, provide more sophisticated features to support modular software, as we shall see later in the case of Ada.
3. One of the great strengths of object orientation is the modularity inherent in the notion of a class.

□ SUPPORT FOR SEPARATE COMPILATION.

1. For small programs, it is common for the entire program to reside in a single file that is compiled as a single unit. But for larger programs, it is almost essential to allow the program to be spread over multiple files (perhaps 1000's) compiled separately. In this way, when a change is made, only the affected file(s) need to be recompiled.

2. Many languages support this by adding a separate step to the program build process called LINKING.

- a. Each source file is compiled to produce an object file.
- b. A separate linking step combines all the object files, together with needed code from libraries, into a single executable file.
- c. Example: a command such as gcc or g++ actually invokes both of these steps.
- d. Of course, the longer the program, the more work the linking step has to do.

3. To make this work, there must be a mechanism whereby a module being compiled can be aware of "public" features of other modules. This is often handled by splitting a module into two files.

- a. Example: the .h and .c/.cc files used by C/C++.
- b. Some languages use a single source file, but the compiler produces two object files - one containing just declaration information and the other the actual code. Other modules need to explicitly import the former, while the linker uses the latter. Examples: Ada, Modula

c. Some languages include declaration information in a single output file produced by compilation that other modules can use via import. Example: Java .class files

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 18

CSC413: Organization of Programming Languages

d. One tricky issue is ensuring that, when the interface of a module is changed, other modules that depend on it are recompiled. There is no totally general solution to this problem, short of a "clean build".

□ DATA TYPES AND DATA STRUCTURING FACILITIES available in

the language. By "Data types and Structures", we mean the ability of a language to support a variety of data values (integers, real, strings, pointers etc.) and non elementary collections of these.

Ideally, the type structure should be EXTENSIBLE, allowing the programmer to easily create and use new data types to fit the problem at hand.

1. FORTRAN is an example of a language that is particularly weak on this score, having only arrays as structured types - no records or pointer variables. Thus, what

would be done with structs/classes in C like languages will have to be done with individual variables in FORTRAN, and linked structures can only be implemented by using arrays of nodes

- dynamic storage allocation is not possible. There is no facility for declaring new data types, either. A number of other languages share this shortfall, including APL and BASIC.

2. Most languages include at least arrays, records, and pointers as structuring facilities. Also, they typically include a data type creation facility, though the operations available on user-created types are limited. Thus, user-created types are "second-class citizens".

3. Object-oriented languages such as Java carry this even further, of course.

4. Some languages (e.g. Ada, C++) even allow the standard operators to be redefined for user-defined data types.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 19

CSC413: Organization of Programming Languages

III. Attributes Relating to Performance

Last on our list - but not unimportant - are criteria relating to how the language performs. First, we consider the performance of language translators. Ideally, the language should lend itself to FAST COMPILATION.

1. In general, the more complex the syntax of the language, the longer a program of a given length will take to compile. This was rather dramatically illustrated by the compilers we had on our PDP-11/70.

Student projects in Pascal and FORTRAN would compile quite quickly.

Programs written in BASIC-PLUS-TWO would take much longer, and COBOL programs of the same length would seem to take forever!

2. When developing large programs, it is nice to have a separate compilation facility that allows the program to be spread over several files. When a small change is made, only the affected file, and perhaps others that depend on it, need to be recompiled. Most current languages support this.

Also important in many cases (but not all) is that the compiler produce EFFICIENT OBJECT CODE.

1. In part, this is a matter of compiler technology; but some language features make this harder.

2. Of course, this goal conflicts with the goal of fast compilation. An optimizing compiler spends extra time during compilation to produce better object code. This is nice for production software, but is not as pleasant during program development. Optimization is the name given to the collection of techniques that compilers may use to decrease the size and/or increase the execution speed of the code they produce. If little or no optimization is done, compilation can be done much faster than if a significant effort is made to produce optimized code.

3. Some languages are supported by two compiler versions - a fast "checkout" compiler that produces less than optimal code, but which can be used during debugging; and an optimizing compiler that is slower but produces production-quality code. Or, a single compiler may include a command line option to turn optimization on or off or even specify the degree of optimization desired - trading off compilation time for execution time.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 20
CSC413: Organization of Programming Languages

quality code. Or, a single compiler may include a command line option to turn optimization on or off or even specify the degree of optimization desired - trading off compilation time for execution time.

Example: PL/I implementations included both a "checkout" compiler and an optimizing compiler.

Example: the gnu C and other compilers include a -O switch with possible values 0 (no optimization) or 1, 2, 3 (increasing degrees of optimization.)

(Unfortunately, sometimes the two compilers or the one compiler with different optimization settings don't process the same language constructs in exactly the same way. Of course, this is more of a compiler problem than a language problem per se)

Last - but by no means least - we consider the matter of MACHINE INDEPENDENCE or PORTABILITY.

1. One of the original reasons for adopting higher level languages was the desire to be able to move a program from one type of machine to another without rewriting it. To some extent, all higher-level languages achieve this goal; but some do much better than others.

2. Most important for portability is the existence of a well-defined and accepted language standard.

a. Many languages have been standardized by formal bodies like ANSI or ISO. For others, the original report by the language author may serve as a standard - though not as strong of a one. Some languages have no clear-cut standard, though.

b. A standard does not help much if different implementers of the language choose to go beyond the standard with various extensions, each in his own way. The classic example of this is BASIC.

Despite the existence of an ANSI standard, no two implementations are the same, due to different extensions. (Actually, some also implement less than the standard.) Thus, it is important that the set of features contained in the standard be complete enough to help implementers resist the temptation to add incompatible extensions.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 21
CSC413: Organization of Programming Languages

c. In the case of Ada, the Department of Defense copyrighted the name Ada, in order to ensure that ALL implementations handle exactly the same language, thus facilitating portability. (In order to use the name Ada, a compiler must pass a validation test that ensures it handles the language exactly as specified.)

3. Standardization by itself is not enough, though, even when the standard is adhered to. Certain characteristics of the underlying machine have a way of showing up unavoidably in the implementation.

a. For example, every machine has a basic word length which determines the range of integers that can be processed by regular machine instructions.

i. Historically, microprocessor systems often used 16 bit integers, restricting the range of integers to -32768 .. 32767.

ii. Many systems today use 32 bits, leading to integers ranging from -2 billion to +2 billion.

iii. Still other systems use 64 bits, leading to integers ranging from -100 trillion to +100 trillion.

iv. A program which relies on the range of integers available on one machine may not run correctly on another machine whose range of values is smaller. (A similar phenomenon arises with the range of values and precision of real numbers.)

b. Again, different machines use different coding schemes to represent characters, with ASCII, EBCDIC, and Unicode having wide use. This can lead to problems, as follows:

i. In ASCII, the codes for alphabetic characters are contiguous, without any gaps. This is not true in EBCDIC. For example, the EBCDIC code for I is 201, while that for J is 209. A program that relies on the letters being contiguous - such as a cipher program - will fail if ported to an EBCDIC machine.

ii. In ASCII, upper case letters have codes 32 less than corresponding lower case letters. On EBCDIC machines, their code is 64 greater! A program that converts between lower and upper case letters may have a problem here.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 22

CSC413: Organization of Programming Languages

c. Java addresses these issues by stipulating as part of the standard that byte, short, int, and long are - respectively - 8, 16, 32 and 64 bit integers; and that characters are represented internally using Unicode.

1.3 Why so Many Programming Languages?

Millions of programming languages have been invented, and several thousands of them are actually in use. Why?

□ Programming languages have evolved over time as better ways have been developed to design them.

o First programming languages were developed in the 1950s. Since then thousands of languages have been developed

o New programming paradigm i.e. new programming styles or way of thinking is evolving

□ Different programming languages are designed for different types of programs

- o not every language is perfect for every task

- Advances in hardware

- Socio-economic factors: proprietary interests, commercial advantage of language developers

However, compared to natural languages that developed and evolved independently, programming languages are far more similar to each other.

This is because:

- o different programming languages share the same mathematical foundation (e.g., Boolean algebra, logic);

- o they provide similar functionality (e.g., arithmetic, logic operations, and text processing);

- o they are based on the same kind of hardware and instruction sets;

- o they have common design goals: find languages that make it simple for humans to use and efficient for hardware to execute

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 23

CSC413: Organization of Programming Languages

2. DOMAINS OF PROGRAMMING LANGUAGES

Computers have been applied to a myriad of different areas, from controlling nuclear power plants to providing video games in mobile phones. Because of this great diversity in computer use, programming languages with very different goals have been developed.

2.1 Scientific Applications

The first digital computers, which appeared in the late 1940s and early 1950s, were invented and used for scientific applications.

Typically, the scientific applications of that time

- o Used relatively simple data structures, but required large numbers of floating-point arithmetic computations.

- o The most common data structures were arrays and matrices;

- o The most common control structures were counting loops selections

The first language for scientific applications was Fortran. ALGOL 60 and most of its descendants were also intended to be used in this area, although they were designed to be used in related areas as well.

2.2 Business Applications

The use of computers for business applications began in the 1950s. Special computers were developed for this purpose, along with special languages. The first successful high-level language for business was COBOL the initial version of which appeared in 1960. It is still the most commonly used language for these applications.

Business languages are characterized by:

- o facilities for producing elaborate reports,

- o precise ways of describing and storing decimal numbers and character data, and
- o the ability to specify decimal arithmetic operations.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 24

CSC413: Organization of Programming Languages

2.3 Artificial Intelligence

Artificial intelligence (AI) is a broad area of computer applications characterized by

- o The use of symbolic rather than numeric computations. Symbolic computation means that symbols, consisting of names rather than numbers, are manipulated.
- o Also, symbolic computation is more conveniently done with linked lists of data rather than arrays.

This kind of programming sometimes requires more flexibility than other programming domains. For example, in some AI applications the ability to create and execute code segments during execution is convenient.

The first widely used programming language developed for AI applications was the functional language LISP which appeared in 1959. Most AI applications developed prior to 1990 were written in LISP or one of its close relatives. More recently, some AI applications have been written in systems languages such as C. Scheme, a dialect of LISP, and Prolog.

2.4 Systems Programming

The operating system and the programming support tools of a computer system are collectively known as its systems software.

- o Systems software is used almost continuously and so it must be efficient.
- o Furthermore, it must have low level features that allow the software interfaces to external devices to be written.

In the 1960s and 1970s, some computer manufacturers, such as IBM, Digital, and Burroughs (now UNISYS), developed special machine-oriented high-level languages for systems software on their machines. For IBM mainframe computers, the language was PL/S, a dialect of PL/I; for Digital, it was BLISS, a language at a level just above assembly language; for Burroughs, it was Extended ALGOL.

- o However, most system software is now written in more general programming languages, such as C and C++.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 25

CSC413: Organization of Programming Languages

2.5 Web Software/Scripting language

The World Wide Web is supported by an eclectic collection of languages, ranging from markup languages, such as HTML, which is not a programming language, to general-purpose programming languages, such as Java.

- o Because of the pervasive need for dynamic Web content, some computation capability is often included in the technology of content presentation.
- o This functionality can be provided by embedding programming code in an HTML document.
- Such code is often in the form of a scripting language, such as JavaScript or PHP

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 26

CSC413: Organization of Programming Languages

3. INFLUENCES ON DESIGN OF PROGRAMMING LANGUAGES

3.1 Computer Architecture

The basic architecture of computers has had a profound effect on language design. Most of the popular languages of the past 50 years have been designed around the prevalent computer architecture, called the von Neumann architecture, after one of its originators, John von Neumann (pronounced “Von Noyman”).

In a von Neumann computer:

- o Data and programs stored in memory
- o Memory is separate from CPU
- o Instructions and data are piped from memory to CPU
- o Variables model memory cells
- o Assignment statements model piping

Languages developed around the prevalent computer architecture, known as the von Neumann architecture is called imperative languages.

The overall structure of a von Neumann computer is shown as follows:

The execution of a machine code program on a von Neumann architecture computer occurs in a process called the fetch-execute cycle.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 27

CSC413: Organization of Programming Languages

The fetch-execute cycle can be simply described by the following algorithm:

```

initialize the program counter
repeat forever
  fetch the instruction pointed to by the program counter
  increment the program counter to point at the next instruction
  decode the instruction
  execute the instruction
end repeat
  
```

The “decode the instruction” step in the algorithm means the instruction is examined to determine what action it specifies.

The speed of the connection between a computer’s memory and its processor usually determines the speed of the computer, because instructions often can be

executed faster than they can be moved to the processor for execution.

- o This connection is called the von Neumann bottleneck; it is the primary limiting factor in the speed of von Neumann architecture computers.

- o The von Neumann bottleneck has been one of the primary motivations for the research and development of parallel computers.

3.2. Programming design Methodologies Influences

All of the following evolutionary steps in software development methodologies led to new language construct to support them.

- The late 1960s and early 1970s brought an intense analysis, begun in large part by the structured-programming movement, of both the software development process and programming language design.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 28

CSC413: Organization of Programming Languages

- o An important reason for this research was the shift in the major cost of computing from hardware to software, as hardware costs decreased and programmer costs increased.

- o In addition, progressively larger and more complex problems were being solved by computers.

- such problems as controlling large petroleum-refining facilities and providing worldwide airline reservation systems.

- The new software development methodologies that emerged as a result of the research of the 1970s were called top-down design and stepwise refinement. The primary programming language deficiencies that were discovered were incompleteness of type checking and inadequacy of control statements (requiring the extensive use of gotos).

- In the late 1970s, a shift from procedure-oriented to data-oriented program design methodologies began.

- o Simply put, data-oriented methods emphasize data design, focusing on the use of abstract data types to solve problems.

- o For data abstraction to be used effectively in software system design, it must be supported by the languages used for implementation. The first language to provide even limited support for data abstraction was SIMULA 67

- The latest step in the evolution of data-oriented software development, which began in the early 1980s, is object-oriented design.

- o Object-oriented methodology begins with data abstraction, which encapsulates processing with data objects and controls access to data, and adds inheritance and dynamic method binding.

- Inheritance is a powerful concept that greatly

enhances the potential reuse of existing software,

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 29

CSC413: Organization of Programming Languages

thereby providing the possibility of significant increases in software development productivity.

o Object-oriented programming developed along with a language that supported its concepts

- Support for object-oriented programming is now part of most popular imperative languages, including Ada 95, Java, C++, and C#.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 30

CSC413: Organization of Programming Languages

4. OVERVIEW OF DIFFERENT PROGRAMMING PARADIGMS

A paradigm, derived from the Greek word paradeiknyai, commonly refers to a new method of thinking about a problem or situation.

It also refers to:

- A mode of thinking aka a programming methodology
- A set of coherent abstractions used to effectively model a problem/domain
- A pattern that serves as a school of thoughts for programming of computers
- The way we express ourselves in a computer program i.e. a programming style
- A set of basic principles, concepts, and methods for how a computation or algorithm is expressed.

The major paradigms we will study are: Structured, imperative, object-oriented, functional, and logic paradigms. Programming languages naturally fall into a number of these fundamental styles or paradigms.

4.1 Structured Programming Paradigm

Programs written in programming languages before the emergence of structured programming concepts were characterized as spaghetti programming or monolithic programming.

The structured programming paradigm proposed methods to logically structure the programs developed into separate smaller, more manageable components.

- o Each component (module) had a single entry and a single exit point.
- o Control was passed downward through the structure without unconditional branches (e.g., goto statements) to higher levels of the structure.
- o Only three types of control structures were used: sequential, conditional branch, and iteration.

Structuring computer programs can make each part more understandable and make large programs easier to maintain.

Examples of structured programming language include Fortran, C, C++, Pascal,

Java, C# etc

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 31

CSC413: Organization of Programming Languages

Characteristics of structured programming languages

- i. Uses procedural abstraction and top-down design to identify program components (also called modules or structures).
- ii. Program control flow is based on decisions, sequences, loops, but does not use goto statements
- iii. Modules are developed, tested separately and then integrated into the whole program

Structured programming languages use the following structures as well as sequence structure

1. Sequence

Stmt1

Stmt2

Stmntn

2. Repetition

While condition

action

End While

3. Selection

If condition Then

action

End If

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 32

CSC413: Organization of Programming Languages

4.2. Imperative/Procedural Programming Paradigm

Procedural programming is a style of programming in which operations are executed one after another in sequence. It expresses computation by fully-specified and fully-controlled manipulation of named data in a stepwise fashion. It uses statements to change a program's state. The stepwise fashion may include:

- o The data or values are initially stored in variables (memory locations),
- o The data or values are taken out of (read from) memory, manipulated in ALU (arithmetic logic unit), and then stored back in the same or different variables (memory locations).
- o Finally, the values of variables are sent to the I/O devices as output.

The foundation of imperative languages is the stored program concept- based on computer hardware organization and architecture of Von Neumann's machine.

Typical imperative programming languages include Assembly languages Fortran,

Algol, Ada, Pascal, C, C++, BASIC, Java, Python etc.

Note: Imperative programming languages may be multi-paradigm and appear in other classifications

Characteristics of Imperative programming languages

- i. Execution of statements (commands) follow a stepwise fashion, hence order of execution is crucial for correct execution
- ii. Program changes state through assignment statements
- iii. Programmers control all aspects: algorithm specification, memory management, variable declarations, etc.
- iv. Abstractions of the way a traditional Von Neumann computer works

The following is Java code snippets illustrates the sequence of imperative language programs

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 33

CSC413: Organization of Programming Languages

```
int age;
Scanner Input = new Scanner (System.in);
System.out.println("Enter age of Candidate:");
age = Input.nextInt();
if( age >= 16 && age <= 25)
System.out.println("Admit as regular Student");
else if( age >= 25 && age <= 60)
System.out.println("Admit as part time Student");
else
System.out.println("No Admisssion");
if (Input != null)
Input.close();
```

4.3. Object-Oriented Programming Paradigm

This programming paradigm is basically the same as the imperative paradigm, except

- o that related variables and operations on variables are organized (encapsulated) into classes of objects.
- o The access privileges of variables and methods (operations) in objects can be defined to reduce (simplify) the interaction among objects.
- o Objects are considered the main building blocks of programs.

Typical object-oriented programming languages include: Smalltalk, C++, Java, and C#.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 34

CSC413: Organization of Programming Languages

Characteristics of Object-Oriented programming

- i. Class definitions – Basic building blocks OOP and a single entity which has

data and operations on data together

ii. Objects – The instances of a class which are used in real functionality – its variables and operations. Data as well as operations are encapsulated in objects

iii. Information hiding is used to protect internal properties of an object

iv. Inheritance and class hierarchy – Classes are organized in inheritance hierarchies and thus enhance reusability and extension of existing classes

v. Message passing – Objects communicate through invoking methods and sending data to them. This feature of sending and receiving information among objects through function parameters is known as Message Passing.

The following is an example of Java codes showing the use of classes and objects.

```
import java.util.Scanner;
public class admMode
public static void main(String [] args)
int age;
Scanner Input = new Scanner (System.in);
System.out.println("Enter age of Candidate:");
age = Input.nextInt();
if( age >= 16 && age <= 25)
System.out.println("Admit as regular Student");
else if( age >= 25 && age <= 60)
System.out.println("Admit as part time Student");
else
System.out.println("No Admission");
if (Input != null)
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 35

CSC413: Organization of Programming Languages

```
Input.close();
```

4.4 Functional/Applicative Programming Paradigm

This expresses computation in terms of mathematical functions. There is no concept of memory locations in functional programming languages.

- o Each function will take a number of values as input (parameters) and produce a single return value (output of the function).

- o The return value cannot be stored for later use. It has to be used either as the final output or immediately as the parameter value of another function.

Functional programming is about defining functions and organizing the return values of one or more functions as the parameters of another function. Functional programming languages are mainly based on the lambda calculus.

Typical functional programming languages include ML, SML, Lisp/Scheme,

Python, Erlang, and Haskell.

Characteristics of Functional Programming (FP) Languages

- i. State does not exist. FP Programs does not contain state. That means all Data is Immutable Data and Functions cannot change state.
- ii. Low importance of Order of Execution: In FP Languages, we write programs with a set of Independent Functions. Functions contain a set of statements.

In FP, the order of execution of those Functions does not have much importance because they do not have state and all Functions work independently. Even if we change the order of execution still they produce the same results.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 36

CSC413: Organization of Programming Languages

- iii. Primary Manipulations Units: In FP Languages, Primary Manipulations units are Functions and Data Structures because all Programs are made up of using these units.
- iv. Primary Flow Controls: FP Languages don't use Flow Controls like For...Loop, Do...While Loop, While...Loop etc and also don't use Conditional statements like If..Else or Switch Statements. All FP Languages write programs using functions and function calls
- v. Variables are Immutable: In functional programming a variable cannot be modified after it has been initialized.

The following are examples of functional programs in Scheme

To find the square of a number:

Method 1: Using Nameless function

```
(LAMBDA (x) (* x x))
```

is a nameless function that returns the square of its given numeric parameter. In Scheme, a nameless function actually includes the word LAMBDA, and is called a lambda expression (anonymous functions).

Method 2: Using a named function

```
(DEFINE (square number) (* number number))
```

After the interpreter evaluates this function, it can be used, as in

```
(square 5)
```

which displays 25.

Note: (square number) - square is the function name, while number is parameter while

(* number number) - constitute the expression

DEFINE binds the name square to a functional expression

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 37

CSC413: Organization of Programming Languages

4.5. Logic/Declarative Programming Paradigm

This paradigm expresses computation in terms of logic predicates. A logic program is a set of facts, rules, and questions.

- o The execution process of a logic program is to compare a question to each fact and rule in the given fact and rule-base.

- o If the question finds a match, we receive a yes answer to the question.

Otherwise, we receive a no answer to the question.

Logic programming is about finding facts, defining rules based on the facts, and writing questions to express the problems we wish to solve.

The particular form of symbolic logic that is used for logic programming is called first-order predicate calculus

There are three basic statements:

- Facts are fundamental assertions about the problem domain (e.g. "Socrates is a man")

- Rules are inferences about facts in the domain (e.g. "All men are mortal.")

- Queries are questions about that domain (e.g. "Is Socrates mortal?")

Prolog is the only significant logic programming language.

Characteristics of Declarative Programming languages

- i. They are based on axioms, inference rules, and queries.

- ii. Program execution becomes a systematic search in a set of facts, making use of a set of inference rules

- iii. Values are assigned to variables by means of automatically generated substitutions

The following are examples of statements in Prolog.

The general form is:

functor(parameter list)

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 38

CSC413: Organization of Programming Languages

Where functor is any atom and is used to identify the structure. The parameter list can be any list of atoms, variables, or other structures. Atoms are the symbolic values of Prolog

female(shelley).

male(bill).

female(mary).

male(jake).

father(bill, jake).

father(bill, shelley).

mother(mary, jake).

mother(mary, shelley).

These simple structures state certain facts about jake, shelley, bill, and mary.

For example,

- o the first states that shelley is a female.
- o the last four connect their two parameters with a relationship that is named in the functor atom; for example, the fifth proposition might be interpreted to mean that bill is the father of jake.

Note that these Prolog propositions, like those of predicate calculus, have no intrinsic semantics. They mean whatever the programmer wants them to mean.

For example, the proposition

father(bill, jake).

could mean bill and jake have the same father or that jake is the father of bill.

The most common and straightforward meaning, however, might be that bill is the father of jake.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 39

CSC413: Organization of Programming Languages

5. PROGRAMMING LANGUAGE IMPLEMENTATION METHODS

Programming languages can be implemented by any of three general methods:

- o compilation,
- o pure interpretation, and
- o hybrid implementation.

5.1. Compilation

Programs can be translated into machine language, which can be executed directly on the computer. This method is called a compiler implementation and

- o It has the advantage of very fast program execution, once the translation process is complete.

- o Most production implementations of languages, such as C, COBOL, C++, and Ada, are by compilers.

The language that a compiler translates is called the source language. The process of compilation and program execution takes place in several phases, the most important of which are shown in Figure 1.

Fig. 1: The Compilation process

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 40

CSC413: Organization of Programming Languages

The lexical analyzer gathers the characters of the source program into lexical units.

- o The lexical units of a program are identifiers, special words, operators, and punctuation symbols.
- o The lexical analyzer ignores comments in the source program because the compiler has no use for them.

The syntax analyzer takes the lexical units from the lexical analyzer:

- o It uses them to construct hierarchical structures called parse trees.
- o These parse trees represent the syntactic structure of the program.
- o In many cases, no actual parse tree structure is constructed;
- o rather, the information that would be required to build a tree is generated and used directly.

The intermediate code generator:

- o produces a program in a different language, at an intermediate level between the source program and the final output of the compiler: the machine language program.
- o Intermediate languages sometimes look very much like assembly languages, and in fact, sometimes are actual assembly languages.

The semantic analyzer is an integral part of the intermediate code generator.

- o The semantic analyzer checks for errors, such as type errors, that are difficult, if not impossible, to detect during syntax analysis.

Optimization, which improves programs (usually in their intermediate code version) by making them smaller or faster or both, is often an optional part of compilation.

- o In most commercial and industrial situations, execution speed is more important than compilation speed, so optimization is routinely desirable.

□ Because many kinds of optimization are difficult to do on machine language, most optimization is done on the intermediate code.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 41

CSC413: Organization of Programming Languages

- o The code generator translates the optimized intermediate code version of the program into an equivalent machine language program.

The symbol table serves as a database for the compilation process.

- o The primary contents of the symbol table are the type and attribute information of each user-defined name in the program.
- o This information is placed in the symbol table by the lexical and syntax analyzers and is used by the semantic analyzer and the code generator.

5.2. Pure Interpretation

With this approach, programs are interpreted by another program called an interpreter, with no translation whatever.

- o The interpreter program acts as a software simulation of a machine whose fetch-execute cycle deals with high-level language program statements rather than machine instructions.

Pure interpretation has the advantage of

- o allowing easy implementation of many source-level debugging operations, because all run-time error messages can refer to source-level units.

□ For example, if an array index is found to be out of range, the error message can easily indicate the source line and the name of the array.

On the other hand, this method has the serious disadvantage

- o Execution is 10 to 100 times slower than in compiled systems.

□ The primary source of this slowness is the decoding of the high-level language statements, which are far more complex than machine language instructions (although there may be fewer statements than instructions in equivalent machine code).

- o Furthermore, regardless of how many times a statement is executed, it must be decoded every time.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 42

CSC413: Organization of Programming Languages

□ Therefore, statement decoding, rather than the connection between the processor and memory, is the bottleneck of a pure interpreter.

- o Another disadvantage of pure interpretation is that it often requires more space.

□ In addition to the source program, the symbol table must be present during interpretation.

□ Furthermore, the source program may be stored in a form designed for easy access and modification rather than one that provides for minimal size.

Examples of pure interpretation languages:

- o early languages of the 1960s (APL, SNOBOL, and LISP) were purely interpreted

- o recent years, JavaScript and PHP are now widely used.

The process of pure interpretation is shown in Figure 2.

Fig. 2: Pure Interpretation

5.3. Hybrid Implementation Systems

Some language implementation systems are a compromise between compilers and pure interpreters;

- o They translate high-level language programs to an intermediate language designed to allow easy interpretation.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 43

CSC413: Organization of Programming Languages

□ This method is faster than pure interpretation because the source language statements are decoded only once.

- o Such implementations are called hybrid implementation systems.

The process used in a hybrid implementation system is shown in Figure 3. Instead of translating intermediate language code to machine code; it simply interprets the intermediate code.

Fig. 3: Hybrid implementation system

Perl is implemented with a hybrid system.

- o Perl programs are partially compiled to detect errors before interpretation and to simplify the interpreter.

Initial implementations of Java were all hybrid.

- o Its intermediate form, called byte code, provides portability to any machine that has a byte code interpreter and an associated run-time system.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 44

CSC413: Organization of Programming Languages

□ Together, these are called the Java Virtual Machine. There are now systems that translate Java byte code into machine code for faster execution.

A Just-in-Time (JIT) implementation system

- o Initially translates programs to an intermediate language.
- o Then, during execution, it compiles intermediate language methods into machine code when they are called.

□ The machine code version is kept for subsequent calls.

JIT systems are now widely used for Java programs.

Also, the .NET languages are all implemented with a JIT system.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 45

CSC413: Organization of Programming Languages

6. THE CONCEPT OF VARIABLES IN PROGRAMMING LANGUAGES

A Variable is:

- o a representation of a memory location to which a unique name is attached to facilitate subsequent referencing
- o an abstraction of a computer memory cell or collection of cells
- o The architecture's two primary components are:
 - its memory, which stores both instructions and data, and
 - its processor, which provides operations for modifying the contents of the memory.
- o The abstractions in a language for the memory cells of the machine are variables.

□ Why Use Variable?□

□ The move from machine languages to assembly languages was largely one of replacing absolute numeric memory addresses for data with names□

□ making programs far more readable and therefore easier to write and maintain.

□ It also provides an escape from the problem of manual absolute addressing□

□ because the translator that converted the names to actual addresses also chose those addresses.

□ Attributes of Variable□

A variable can be characterized as a sextuple of attributes:□

- o Name
- o Address
- o Value
- o Type
- o Lifetime
- o Scope

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 46

CSC413: Organization of Programming Languages

6.1. NAME□

A name is a string of characters used to identify some entity in a program.

□ Design Issues□

The following are the primary design issues for names:

- Are names case sensitive?
- Are the special words of the language reserved words or keywords?

A. Name Forms□

The earliest programming languages used single-character names.□

o This notation was natural because early programming was primarily mathematical, and mathematicians have long used single-character names for unknowns in their formal notations.

□ FORTRAN I broke this tradition of the single-character name, allowing up to six characters in its names. Fortran 95+ (i.e. version 95 and above) allows up to 31 characters in its names.□

□ Names in Java, C#, and Ada have no length limit, and all characters in them are significant. C++ does not specify a length limit on names, although implementers sometimes do.□

Names in most programming languages have the same form:□

o a letter followed by a string consisting of letters, digits, and underscore characters (_).

o Although the use of underscore characters to form names was widely used in the 1970s and 1980s, that practice is now far less popular.

o In the C-based languages, it has to a large extent been replaced by the

so-called camel notation, in which all of the words of a multiple-word name except the first are capitalized, as in `dateOfBirth` (It is

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 47

CSC413: Organization of Programming Languages

called “camel” because words written in it often have embedded uppercase letters, which look like a camel’s humps).

In many languages, notably the C-based languages (e.g. C, C++, Java, and C#), uppercase and lowercase letters in names are distinct; that is, names in these languages are case sensitive.□

o case sensitivity violates the design principle (e.g. uniformity) that language constructs that look similar should have similar meanings.

B. Special Words□

These are words with special meanings ascribed to them by the syntax of a language□

In most languages, special words are classified as□

- o reserved words, which means they cannot be redefined by programmers
- o but in some they are only keywords, which means they can be redefined.

□ keywords□

A keyword is a word of a programming language that is special only in certain contexts.□

o Fortran is the only remaining widely used language whose special words are keywords.

o In Fortran, the word `Integer`, when found at the beginning of a statement and followed by a name, is considered a keyword that indicates the statement is a declarative statement.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 48

CSC413: Organization of Programming Languages

o However, if the word `Integer` is followed by the assignment operator, it is considered a variable name.

e.g.

`Integer Apple`

`Integer = 4`

□ A reserved word□

is a special word of a programming language that cannot be used as a name.□

o As a language design choice, reserved words are better than keywords because the ability to redefine keywords can be confusing.□

o For example, in Fortran, one could have the following statements:

Integer Real

Real Integer

□ These statements declare the program variable Real to be of Integer type and the variable Integer to be of Real type.□

- It could be misleading to program readers.□

6.2. ADDRESS□

The address of a variable is the machine memory address with which it is associated.□

The address of a variable is sometimes called its L-value□

o because the address is what is required when the name of a variable appears in the left side of an assignment.

It is possible to have multiple variables that have the same address.□

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 49

CSC413: Organization of Programming Languages

o When more than one variable name can be used to access the same memory location, the variables are called aliases.

□ The problems of using Aliases□

o Aliasing is a hindrance to readability because it allows a variable to have its value changed by an assignment to a different variable.

o For example, if variables named total and sum are aliases, any change to the value of total also changes the value of sum and vice versa.

o A reader of the program must always remember that total and sum are different names for the same memory cell.

o Because there can be any number of aliases in a program, this may be very difficult in practice.

o Aliasing also makes program verification more difficult□

□ Creating Aliases□

One common way of creating aliases in C and C++ is with their union types.

e.g.

```
int n = 10;
```

```
int & r = n; // r is a reference for n, an alias
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 50

CSC413: Organization of Programming Languages

NOTE: & reference the address of a variable

An example of alias in java:

```

B[] b = new B[10];
A[] a = b;
a[0] = new B();
b[0].methodParent();// methodParent is a defined method.

```

6.3. TYPE

The type of a variable determines the range of values the variable can store and the set of operations that are defined for values of the type.

For example the int type in Java

- o specifies a value range of -2147483648 to 2147483647

- o and arithmetic operations for addition, subtraction, multiplication, division, and modulus.

6.4. VALUE

The value of a variable is the contents of the memory cell or cells associated with the variable.

- A variable's value is sometimes called its r-value because it is what is required when the name of the variable appears in the right side of an assignment statement.

- To access the r-value, the l-value (i.e. the address) must be determined first.

Example:

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 51

CSC413: Organization of Programming Languages

```
sum = num * 3;
```

sum is the l-value, which is the address that stores the value of evaluating the expression(num * 3).

It is convenient to think of computer memory in terms of abstract cells, rather than physical cells

- The physical cells, or individually addressable units, of most contemporary computer memories are byte-size - eight bits in length.

- This size is too small for most program variables.

- An abstract memory cell has the size required by the variable with which it is associated.

- o For example, although floating-point values may occupy four physical bytes in a particular implementation of a particular language,

- o a floating-point value is thought of as occupying a single abstract memory cell.

6.5. LIFETIME

The lifetime of a variable is the time during which the variable is bound to

a specific memory location.□

o So, the lifetime of a variable begins when it is bound to a specific cell and ends when it is unbound from that cell.

□ A binding is an association between an attribute and an entity,□

o such as between a variable and its type or value, or between an operation and a symbol.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 52

CSC413: Organization of Programming Languages

□ The time at which a binding takes place is called binding time.□

o Examples of Binding & their Binding Times.□

o Language design time – operators bind to operations e.g. * bound to multiplication

o Language implementation time – in C family, int is bound to a range of possible values e.g. -32,768 to 32,767 (for 16 bit machines i.e. 2 bytes)

o Compile time – A variable bound to a particular data type

o Load time – a variable bound to storage cell when program is loaded into memory□

o Link time – a call to a library subprogram is bound to the subprogram code at link time

o Run time – declared variables are bound at run time

□ Types of Binding□

• Static - A binding is static if it first occurs before run time begins and remains unchanged throughout program execution. e.g.

`float a[20];` //a is a constant pointer to block of 20 floats.

Also this:

`float* const p = new float[20];` //p is static

• Dynamic - the binding first occurs during run time or can change in the course of program execution.

e.g.

`float* p = new float [20];` //dynamic

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 53

CSC413: Organization of Programming Languages

□ Valuable Applications of Static Variables□

Statically bound variables have several valuable applications in programming.

• Globally accessible variables are often used throughout the execution of a program, thus making it necessary to have them

bound to the same storage during that execution.

- Sometimes it is convenient to have subprograms that are history sensitive. Such a subprogram must have local static variables.

6.6. SCOPE

The scope of a variable is the range of statements in which the variable is visible i.e. it describes where in a program's text, the variable may be used.

□ A variable is visible in a statement if it can be referenced in that statement.□

□ A variable is local in a program unit or block if it is declared there. □

□ The nonlocal variables of a program unit or block are those that are visible within the program unit or block but are not declared there.□

o Global variables are a special category of nonlocal variables.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 54

CSC413: Organization of Programming Languages

Example to illustrate Scoping□

Main ()

x = 11; // not in scope of x

int x;

x = 22; //scope of x

y = 10; //not in scope of y

int y;

y = 15; //scope of y

x = 20; // scope of x

y = 12; //not in scope of y

□ NOTE

Sometimes the scope and lifetime of a variable appear to be related. For example, consider a variable that is declared in a Java method that contains no method calls.

o The scope of such a variable is from its declaration to the end of the method.

o The lifetime of that variable is the period of time beginning when the method is entered and ending when execution of the method terminates.

Although the scope and lifetime of the variable are clearly not the same,

o because static scope is a textual, or spatial (space), concept

o whereas lifetime is a temporal (time) concept, they at least appear to be related in this case.

This apparent relationship between scope and lifetime does not hold in other situations□

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 55

CSC413: Organization of Programming Languages

In C and C++, for example, a variable that is declared in a function using the specifier `static` is statically bound to the scope of that function and is also statically bound to storage.

o So, its scope is static and local to the function, but its lifetime extends over the entire execution of the program of which it is a part.

Scope and lifetime are also unrelated when subprogram calls are involved.

Consider the following C++ functions:

```
void printhead() {
    . . .
} /* end of printhead */
void compute() {
    int sum;
    . . .
    printhead();
} /* end of compute */
```

The scope of the variable `sum` is completely contained within the `compute` function.

o It does not extend to the body of the function `printhead`, although `printhead` executes in the midst of the execution of `compute`.

o However, the lifetime of `sum` extends over the time during which `printhead` executes.

Whatever storage location `sum` is bound to before the call to `printhead`, that binding will continue during and after the execution of `printhead`.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 56

CSC413: Organization of Programming Languages

7. DATA TYPES IN PROGRAMMING LANGUAGES

A data type defines a collection of data values and a set of predefined operations on those values□

□ □

□ Categories:□

- 1. Primitive Data Type
- 2. Character String type
- 3. User Defined type
- 4. Array type
- 5. Record type

7.1 Primitive Data Types

□ Data types that are not defined in terms of other types are called primitive data types.□

- Numeric Data Type

- Integer
- Floating point
- Decimal
- Boolean
- Character type

□ Nearly all programming languages provide a set of primitive data types.□

7.1.1 Numeric Data Types□

A. INTEGER□

The most common primitive numeric data type is integer

□ Java includes four signed integer sizes: byte, short, int, and long.[Find out the values for each]

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 57

CSC413: Organization of Programming Languages

□ Some languages, for example, C++ and C#, include unsigned integer types, which are simply types for integer values without signs. Unsigned types are often used for binary data.

□ A signed integer value is represented in a computer by a string of bits

B. FLOATING POINT□

o models real numbers, but the representations are only approximations for many real values.

o Floating-point values are represented as fractions and exponents, a form that is borrowed from scientific notation.

C. DECIMAL□

- Decimal data types store a fixed number of decimal digits, with the decimal point at a fixed position in the value.
- Decimal types have the advantage of being able to precisely store decimal values, at least those within a restricted range, which cannot be done with floating-point.
- Decimal data types are the primary data types for business data processing and are therefore essential to COBOL. C# and F# (pronounced eff Sharp) also have decimal data types.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 58

CSC413: Organization of Programming Languages

□ The disadvantages of decimal types are that

- o the range of values is restricted because no exponents are allowed□

- o their representation in memory is mildly wasteful□

- It takes at least four bits to code a decimal digit.

Therefore, to store a six-digit coded decimal number requires 24 bits of memory.

7.1.2. Boolean Data Types□

Boolean types are perhaps the simplest of all types.□

- Their range of values has only two elements: one for true and one for false.□

- They were introduced in ALGOL 60 and have been included in most general-purpose languages designed since 1960.□

- Boolean types are often used to represent switches or flags in programs.□

7.1.3. Character Data Types□

- Character data are stored in computers as numeric coding.□

- Traditionally, the most commonly used coding was the 8-bit code ASCII (American Standard Code for Information Interchange),□

- which uses the values 0 to 127 to code 128 different characters.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 59

CSC413: Organization of Programming Languages

- Because of the globalization of business and the need for computers to communicate with other computers around the world, the ASCII character set became inadequate.□

- In response, in 1991, the Unicode Consortium published the UCS-2 standard, a 16-bit character set.□

- o This character code is often called Unicode. Unicode includes the characters from most of the world's natural languages.

- The first 128 characters of Unicode are identical to those of ASCII.□

- o Java was the first widely used language to use the Unicode character set. Since then, it has found its way into JavaScript, Python, Perl, C#, and F#.

7.2. Character String Type□

- A character string type is one in which the values consist of

sequences of characters.□

□ Character string constants are used to label output, and the input and output of all kinds of data are often done in terms of strings.□

□ Of course, character strings also are an essential type for all programs that do character manipulation.□

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 60

CSC413: Organization of Programming Languages

□ Design Issues□

The two most important design issues that are specific to character string types are the following:□

- o Should strings be simply a special kind of character array or a primitive type?

- o Should strings have static or dynamic length?

□ Design Issue 1□

- If strings are not defined as a primitive type, string data is usually stored in arrays of single characters and referenced as such in the language.

- This is the approach taken by C and C++.

- C and C++ use char arrays to store character strings.

e.g.

```
char str[] = "apples";
```

□ Design Issue 2 : String Length Options□

□ Static Length string

- o set when the string is created

- o This is the choice for the strings of

- Python, Java's String class, C++ standard class library, Ruby's built-in String class, and the .NET class library available to C# and F#.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 61

CSC413: Organization of Programming Languages

□ □

□ Limited dynamic length string - allow strings to have varying length up to a declared and fixed maximum set by the variable's definition

- o as exemplified by the strings in C and the C-style strings of C++.

- o Recall that strings in C use a special character to

indicate the end of the string's characters, rather than maintaining the string length.

- Dynamic length string - allow strings to have varying length with no maximum

- o as in JavaScript, Perl, and the standard C++ library.

- Ada 95+ supports all three string length options.

7.3. User-Defined Ordinal Type

- An ordinal type is one in which the range of possible values can be easily associated with the set of positive integers.□

- In Java, for example, the primitive ordinal types are integer, char, and boolean.□

- There are two user-defined ordinal types that have been supported by programming languages: enumeration and subrange.□

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 62

CSC413: Organization of Programming Languages

7.3.1 Enumeration Type□

An enumeration type is one in which all of the possible values, which are named constants, are provided, or enumerated, in the definition.

- o Enumeration types provide a way of defining and grouping collections of named constants, which are called enumeration constants.

The definition of a typical enumeration type is shown in the following C# example:□

```
enum days {Mon, Tue, Wed, Thu, Fri, Sat, Sun};
```

- Design issues for enumeration types

- Is an enumeration constant allowed to appear in more than one type definition, and if so, how is the type of an occurrence of that constant in the program checked?

- Are enumeration values coerced to integer?

- Are any other types coerced to an enumeration type?

All of these design issues are related to type checking.

- o If an enumeration variable is coerced to a numeric type, then there is little control over its range of legal operations or its range of values.

- o If an int type value is coerced to an enumeration type, then an enumeration type variable could be assigned any integer value, whether it represented an enumeration constant or not.

C and Pascal were the first widely used languages to include an

enumeration data type. C++ includes C's enumeration types.□

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 63

CSC413: Organization of Programming Languages

In C++, we could have the following:

```
enum colors {red, blue, green, yellow, black};
```

```
colors myColor = blue, yourColor = red;
```

The colors type uses the default internal values for the enumeration constants, 0, 1, . . . , although the constants could have been assigned any integer literal (or any constant-valued expression).

- o The enumeration values are coerced to int when they are put in integer context.

- o This allows their use in any numeric expression.

For example, if the current value of myColor is blue, then the expression myColor++

would assign green to myColor.

Interestingly, none of the relatively recent scripting kinds of languages include enumeration types. These include Perl, JavaScript, PHP, Python, Ruby etc

7.4. Array Type□

- An array is a homogeneous aggregate of data elements in which an individual element is identified by its position in the aggregate, relative to the first element.□

- The individual data elements of an array are of the same type.□

- References to individual array elements are specified using subscript expressions. □

- In many languages, such as C, C++, Java, Ada, and C#, all of the elements of an array are required to be of the same type.□

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 64

CSC413: Organization of Programming Languages

□ Design Issues□

The primary design issues specific to arrays are the following:□

- o What types are legal for subscripts?
- o Are subscripting expressions in element references range checked?
- o When are subscript ranges bound?
- o When does array allocation take place?
- o Are jagged or rectangular multi-dimensioned arrays allowed, or both?
- o Can arrays be initialized when they have their storage allocated?

- o What kinds of slices are allowed, if any?

□ Arrays and Indices

Specific elements of an array are referenced by means of a two-level syntactic mechanism:

- o where the first part is the aggregate name, and
- o the second part is a possibly dynamic selector consisting of one or more items known as subscripts or indices.

The selection operation can be thought of as a mapping from the array name and the set of subscript values to an element in the aggregate. Symbolically, this mapping can be shown as

array_name(subscript_value_list) element

The syntax of array references is fairly universal:

- o The array name is followed by the list of subscripts, which is surrounded by either parentheses or brackets.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 65

CSC413: Organization of Programming Languages

- o In some languages that provide multidimensioned arrays as arrays of arrays, each subscript appears in its own brackets.

A problem with using parentheses to enclose subscript expressions is that they often are also used to enclose the parameters in subprogram calls; this use makes references to arrays appear exactly like those calls.

For example, consider the following Ada assignment statement:

```
Sum := Sum + B(I);
```

Because parentheses are used for both subprogram parameters and array subscripts in Ada, both program readers and compilers are forced to use other information to determine whether B(I) in this assignment is a function call or a reference to an array element.

- o This results in reduced readability.

The designers of Ada specifically chose parentheses to enclose subscripts so there would be uniformity between array references and function calls in expressions, in spite of potential readability problems.

They made this choice in part because both array element references and function calls are mappings.

- o Array element references map the subscripts to a particular element of the array.

- o Function calls map the actual parameters to the function definition and, eventually, a functional value.

Most languages other than Fortran and Ada use brackets to delimit their array indices.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 66

CSC413: Organization of Programming Languages

□ Array Initialization

- o Some languages provide the means to initialize arrays at the time their storage is allocated.
- o In Fortran 95+, an array can be initialized by assigning it an array aggregate in its declaration.
- o An array aggregate for a single-dimensioned array is a list of literals delimited by parentheses and slashes.

For example, we could have

```
Integer, Dimension (3) :: List = (/0, 5, 5/)
```

C, C++, Java, and C# also allow initialization of their arrays, but with one new twist: In the C declaration

```
int list [ ] = {4, 5, 7, 83};
```

the compiler sets the length of the array.

Character strings in C, and C++ are implemented as arrays of char. These arrays can be initialized to string constants, as in

```
char name [ ] = "freddie";
```

The array name will have eight elements, because all strings are terminated with a null character (zero), which is implicitly supplied by the system for string constants.

Arrays of strings in C and C++ can also be initialized with string literals. In this case, the array is one of pointers to characters.

For example,

```
char *names [ ] = {"Bob", "Jake", "Darcie"};
```

For example, names[0] is a pointer to the letter 'B'

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 67

CSC413: Organization of Programming Languages

in the literal character array that contains the characters 'B', 'o', 'b', and the null character.

In Java, similar syntax is used to define and initialize an array of references to String objects.

For example:

```
String[] names = {"Bob", "Jake", "Darcie"};
```

□ Jagged Vs Rectangular Arrays□

□ A rectangular array is a multi-dimensioned array in which all of the rows have the same number of elements and all of the columns have the same number of elements.□

□ A Jagged array is one in which the lengths of the rows need not be

the same. For example, a jagged matrix may consist of three rows, one with 5 elements, one with 7 elements, and one with 12 elements. This also applies to the columns and higher dimensions.□

□ C, C++, and Java support jagged arrays but not rectangular arrays.□

□ Fortran, Ada, C#, and F# support rectangular arrays. (C# and F# also support jagged arrays.)□

C, C++, and Java support jagged arrays but not rectangular arrays.

o In those languages, a reference to an element of a multi-dimensioned array uses a separate pair of brackets for each dimension.

For example,

```
myArray[3][7]
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 68

CSC413: Organization of Programming Languages

Fortran, Ada, C#, and F# support rectangular arrays. (C# and F# also support jagged arrays.)

o In these cases, all subscript expressions in references to elements are placed in a single pair of brackets.

For example,

```
myArray[3, 7]
```

7.5. Record Type□

A record is an aggregate of data elements in which the individual elements are identified by names and accessed through offsets from the beginning of the structure.□

o The elements of a heterogeneous array are all references to data record are of potentially different sizes and reside in adjacent memory locations.□

o In C, C++, and C#, records are supported with the struct data type.□

o In Java and C#, records can be defined as data classes, with nested records defined as nested classes. Data members of such classes serve as the record fields.□

□ Design Issues□

□ What is the syntactic form of references to fields?

□ Are elliptical references allowed?

□ Definitions of Records

The fundamental difference between a record and an array is that record elements, or fields, are not referenced by indices. Instead, the fields are named with identifiers, and references to the fields are made using these identifiers.

In Java and C#, records can be defined as data classes, with nested records defined

as nested classes. Data members of such classes serve as the record fields.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 69

CSC413: Organization of Programming Languages

For example, consider the following declaration:

```
employee.name = "Osazuwa"
```

```
employee.hourlyRate = 50.20
```

These assignment statements create a table (record) named employee with two elements (fields) named name and hourlyRate, both initialized.

Records are used when:

- collection of data values is heterogeneous.
- values are not precessed in the same way.

7.6 Type checking

Type checking is the activity of ensuring that the operands of an operator are of compatible types.

o A compatible type is one that either is legal for the operator or is allowed under language rules to be implicitly converted by compiler-generated code (or the interpreter) to a legal type.

- This automatic conversion is called a coercion.
- For example, if an int variable and a float variable are added in Java, the value of the int variable is coerced to float and a floating-point add is done.

Type error

A type error is the application of an operator to an operand of an inappropriate type. For example, in the original version of C, if an int value was passed to a function that expected a float value, a type error would occur (because compilers for that language did not check the types of parameters).

If all bindings of variables to types are static in a language, then type checking can nearly always be done statically.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 70

CSC413: Organization of Programming Languages

o Dynamic type binding requires type checking at run time, which is called dynamic type checking.

Some languages, such as JavaScript and PHP, because of their dynamic type binding, allow only dynamic type checking.

- o It is better to detect errors at compile time than at run time, because the earlier correction is usually less costly.
- o The penalty for static checking is reduced programmer flexibility.

Strong Typing

A programming language is strongly typed if type errors are always detected. This

requires that the types of all operands can be determined, either at compile time or at run time.

- o The importance of strong typing lies in its ability to detect all misuses of variables that result in type errors.

- o A strongly typed language also allows the detection, at run time, of uses of the incorrect type values in variables that can store values of more than one type.

C and C++ are not strongly typed languages because both include union types, which are not type checked.

ML is strongly typed, even though the types of some function parameters may not be known at compile time. F# is strongly typed.

Java and C#, although they are based on C++, are nearly strongly typed.

Effect of coercion on Typing

However, the coercion also results in a loss of one of the benefits of strong typing— error detection.

- o For example, suppose a program had the int variables a and b and the float variable d. Now, if a programmer meant to type $a + b$, but mistakenly typed

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 71

CSC413: Organization of Programming Languages

$a + d$, the error would not be detected by the compiler. The value of a would simply be coerced to float.

- o So, the value of strong typing is weakened by coercion. Languages with a great deal of coercion, like C, and C++, are less reliable than those with no coercion, such as ML and F#.

Exercises

- What are all of the differences between the enumeration types of C++ and those of Java?
- Multidimensional arrays can be stored in row major order, as in C++, or in column major order, as in Fortran. Develop the access functions for both of these arrangements for two-dimensional arrays.
- How does a decimal value waste memory space?
- For what types of A and B is the simple assignment statement $A = B$ legal in C++ but not Java?
- Why is Java not strongly typed?

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 72

CSC413: Organization of Programming Languages

8. CONTROL STRUCTURES

Computations in imperative-language programs are accomplished by evaluating expressions and assigning the resulting values to variables. However, during program execution, there is flow of control among the different program

statements. Such control provides the mechanisms necessary to make the computations in programs flexible and powerful. Statements that provide such mechanism are called control statements.

Control statements therefore provide:

- o some means of selecting among alternative control flow paths (of statement execution) and
- o some means of causing the repeated execution of statements or sequences of statements.

A control statement and the collection of statements whose execution it controls is called control structure.

The following control structures will be considered:

- i. Selection statement
- ii. Iterative statement

8.1 Selection Statements

A selection statement provides the means of choosing between two or more execution paths in a program. Such statements are fundamental and essential parts of all programming languages

Selection statements fall into two general categories:

- o two-way selection
- o n-way, or multiple selection.

□ Two-Way Selection Statements

Although the two-way selection statements of contemporary imperative languages are quite similar, there are some variations in their designs. The general form of a two-way selector is as follows:

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 73

CSC413: Organization of Programming Languages

if control_expression

then clause

else clause

Design Issues

The design issues for two-way selectors can be summarized as follows:

- o What is the form and type of the expression that controls the selection?
- o How are the then and else clauses specified?
- o How should the meaning of nested selectors be specified?

The Control Expression

Control expressions are specified in parentheses

o if the then reserved word (or some other syntactic marker) is not used to introduce the then clause.

o In those cases where the then reserved word (or alternative marker) is used, there is less need for the parentheses, so they are often omitted, as in Ruby.

In some languages (C, C++, Python), either arithmetic or Boolean expressions can be used. In other contemporary languages, only Boolean expressions can be used for control expressions.

Clause Form

In many languages, the then and else clauses appear as either single statements or compound statements.

- o One variation of this is Perl, in which all then and else clauses must be compound statements, even if they have only one statement.
- o Many languages use braces to form compound statements, which serve as the bodies of then and else clauses.
- o In Python and Ruby, the then and else clauses are statement sequences, rather than compound statements.

□ The complete selection statement is terminated in these languages with a reserved word.

Python uses indentation to specify compound statements.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 74

CSC413: Organization of Programming Languages

For example,

```
if x > y :
x = y
print "case 1"
```

All statements equally indented are included in the compound statement.

Notice that rather than then, a colon is used to introduce the then clause in Python.

Nesting Selectors

Consider the grammar:

```
<if_stmt> → if <logic_expr> then <stmt>
| if <logic_expr> then <stmt> else <stmt>
```

This grammar is ambiguous. The issue is that when a selection statement is nested in the then clause of a selection statement, it is not clear to which if an else clause should be associated.

This problem is reflected in the semantics of selection statements.

Consider the following Java-like code:

```
if (sum == 0)
if (count == 0)
result = 0;
else
result = 1;
```

This statement can be interpreted in two different ways, depending on whether the else clause is

- o matched with the first then clause or

- o matched with the second.

Notice that the indentation seems to indicate that the else clause belongs with the first then clause.

- o However, with the exceptions of Python and F#, indentation has no effect on semantics in contemporary languages and is therefore ignored by their compilers

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 75

CSC413: Organization of Programming Languages

The crux of the problem in this example is that the else clause follows two then clauses with no intervening else clause, and there is no syntactic indicator to specify a matching of the else clause to one of the then clauses.

To force the alternative semantics in Java, the inner if is put in a compound, as in

```
if (sum == 0) {
  if (count == 0)
    result = 0;
  else
    result = 1;
}
```

C, C++, and C# have the same problem as Java with selection statement nesting.

In Perl, the previous code would be written as follows:

Another way to avoid the issue of nested selection statements is to use an alternative means of forming compound statements.

Consider the syntactic structure of the Java if statement. The then clause follows the control expression and the else clause is introduced by the reserved word else.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 76

CSC413: Organization of Programming Languages

- o When the then clause is a single statement and the else clause is present, although there is no need to mark the end, the else reserved word in fact marks the end of the then clause.

- o When the then clause is a compound, it is terminated by a right brace.

- o However, if the last clause in an if, whether then or else, is not a compound, there is no syntactic entity to mark the end of the whole selection statement.

□ Multiple-Selection Statements

The multiple- selection statement allows the selection of one of any number of statements or statement groups. It is, therefore, a generalization of a selector.

Design Issues

The following is a summary of these design issues:

- o What is the form and type of the expression that controls the selection?
- o How are the selectable segments specified?
- o Is execution flow through the structure restricted to include just a single

selectable segment?

o How are the case values specified?

o How should unrepresented selector expression values be handled, if at all?

Examples of Multiple Selectors

1. The switch statement

The C multiple-selector statement, `switch`, which is also part of C++, Java, and JavaScript, is a relatively primitive design.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 77

CSC413: Organization of Programming Languages

Its general form is

```
switch (expression) {
case constant_expression1:statement1;
...
```

```
case constantn: statementn;
```

```
[default: statementn+1]
```

where the control expression and the constant expressions are some discrete type.

This includes

- integer types,
- characters , and
- enumeration types.

The selectable statements can be statement sequences, compound statements, or blocks.

The optional default segment is for unrepresented values of the control expression.

If the value of the control expression is not represented and no default segment is present, then the statement does nothing.

The `switch` statement does not provide implicit branches at the end of its code segments. This allows control to flow through more than one selectable code segment on a single execution.

Consider the following example:

```
switch (index) {
case 1:
case 3: odd += 1;
sumodd += index;
case 2:
case 4: even += 1;
sumeven += index;
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 78

CSC413: Organization of Programming Languages

```
default: printf("Error in switch, index = %d\n",
```

index);

Occasionally, it is convenient to allow control to flow from one selectable code segment to another.

- o The break statement, which is actually a restricted goto, is normally used for exiting switch statements.

- o The break statement transfers control to the first statement after the compound statement in which it appears.

The following switch statement uses break to restrict each execution to a single selectable segment:

```
switch (index) {
case 1:
case 3: odd += 1;
sumodd += index;
break;
case 2:
case 4: even += 1;
sumeven += index;
break;
default: printf("Error in switch, index = %d\n",
index);
```

2. The else-if clause

In many situations, a switch or case statement is inadequate for multiple selection (Ruby's case is an exception). For example, when selections must be made on the basis of a Boolean expression rather than some ordinal type, nested two-way selectors can be used to simulate a multiple selector.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 79
CSC413: Organization of Programming Languages

Consider the following Python selector statement (note that else-if is spelled elif in Python):

```
if count < 10 :
bag1 = True
elif count < 100 :
bag2 = True
elif count < 1000 :
bag3 = True
```

Next example is a java code illustrating else-if selector statement

```
int i = 25;
if(i % 2 == 0) {
System.out.println("i is even");
```

```

else if(i % 3 == 0) {
System.out.println("i is odd, but divisible by 3");
else if(i % 5 == 0) {
System.out.println("i is divisible by 5");
else {
System.out.println("i is not divisible by 2, 3, or
5");

```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 80

CSC413: Organization of Programming Languages

8.2 Iterative Statements

An iterative statement is one that causes a statement or collection of statements to be executed zero, one, or more times. An iterative statement is often called a loop. Every programming language has included some method of repeating the execution of segments of code.

Iteration is the very essence of the power of the computer.

- o If some means of repetitive execution of a statement or collection of statements were not possible, programmers would be required to state every action in sequence;
- o Useful programs would be huge and inflexible and take unacceptably large amounts of time to write and mammoth amounts of memory to store.

Several categories of iteration control statements have been developed.

The primary categories are defined by how designers answered two basic design questions:

- How is the iteration controlled?
- Where should the control mechanism appear in the loop statement?

The primary possibilities for iteration control are logical, counting, or a combination of the two.

The main choices for the location of the control mechanism are

- o the top of the loop or
- o the bottom of the loop.
- o A third option allows the user to decide where to put the control, at the top, at the bottom, or even within the controlled segment.

The body of an iterative statement is the collection of statements whose execution is controlled by the iteration statement.

- We use the term pre-test to mean that the test for loop completion occurs before the loop body is executed and

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 81

CSC413: Organization of Programming Languages

- Post-test to mean that it occurs after the loop body is executed. The iteration statement and the associated loop body together form an iteration

statement.

□ Counter- Controlled Loops

A counting iterative control statement has

- o a variable, called the loop variable, in which the count value is maintained.
- o It also includes some means of specifying the initial and terminal values of the loop variable, and
- o the difference between sequential loop variable values, often called the stepsize.

The initial, terminal, and stepsize specifications of a loop are called the loop parameters.

Design Issues

The following is a summary of these design issues:

- What are the type and scope of the loop variable?
- Should it be legal for the loop variable or loop parameters to be changed in the loop, and if so, does the change affect loop control?
- Should the loop parameters be evaluated only once, or once for every iteration?

The for Statement of the C-Based Languages

The general form of C's for statement is

```
for (expression_1; expression_2; expression_3)
loop body
```

The loop body can be a single statement, a compound statement, or a null statement.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 82

CSC413: Organization of Programming Languages

Because assignment statements in C produce results and thus can be considered expressions, the expressions in a for statement are often assignment statements.

- o The first expression is for initialization and is evaluated only once, when the for statement execution begins.
- o The second expression is the loop control and is evaluated before each execution of the loop body.
- The last expression in the for is executed after each execution of the loop body. It is often used to increment the loop counter.

As is usual in C, a zero value means false and all nonzero values mean true. Therefore, if the value of the second expression is zero, the for is terminated; otherwise, the loop body statements are executed.

Following is an example of a skeletal C for statement:

```
for (count = 1; count <= 10; count++)
...

```

The for statement of C99 and C++ differs from that of earlier versions of C in two ways.

- o First, in addition to an arithmetic expression, it can use a Boolean expression for loop control.
- o Second, the first expression can include variable definitions.

For example,

```
for (int count = 0; count < len; count++) { . . . }
```

The scope of a variable defined in the for statement is from its definition to the end of the loop body.

The for statement of Java and C# is like that of C++, except that the loop control expression is restricted to boolean.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 83

CSC413: Organization of Programming Languages

In all of the C-based languages, the last two loop parameters are evaluated with every iteration.

Furthermore, variables that appear in the loop parameter expression can be changed in the loop body. Therefore, these loops can be complex and potentially unreliable.

□ Logically Controlled Loops

In many cases, collections of statements must be repeatedly executed, but the repetition control is based on a Boolean expression rather than a counter. For these situations, a logically controlled loop is convenient. Actually, logically controlled loops are more general than counter-controlled loops. Every counting loop can be built with a logical loop, but the reverse is not true. Also, recall that only selection and logical loops are essential to express the control structure of any flowchart.

Design Issues

Because they are much simpler than counter-controlled loops, logically controlled loops have fewer design issues.

- o Should the control be pre-test or post-test?
- o Should the logically controlled loop be a special form of a counting loop or a separate statement?

Examples

The C-based programming languages include both pre-test and post-test logically controlled loops that are not special forms of their counter-controlled iterative statements.

The pre-test and post-test logical loops have the following forms:

```
while (control_expression)
```

```
loop body
```

```
and
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 84

CSC413: Organization of Programming Languages

```
do
loop body
while (control_expression);
```

It is legal in both C and C++ to branch into both while and do loop bodies.

The C89 version uses an arithmetic expression for control; in C99 and C++, it may be either arithmetic or Boolean.

Java's while and do statements are similar to those of C and C++, except

- the control expression must be boolean type, and
- the loop bodies cannot be entered anywhere except at their beginnings.

□ User-Located Loop Control Mechanisms

In some situations, it is convenient for a programmer to choose a location for loop control other than the top or bottom of the loop body. As a result, some languages provide this capability. A syntactic mechanism for user-located loop control can be relatively simple, so its design is not difficult. Such loops have the structure of infinite loops but include one or more user-located loop exits.

Design issues

The design issues for such a mechanism are the following:

- o Should the conditional mechanism be an integral part of the exit?
- o Should only one loop body be exited, or can enclosing loops also be exited?

C, C++, Python, Ruby, and C# have unconditional unlabeled exits (break).

Java and Perl have unconditional labeled exits (break in Java, last in Perl).

Following is an example of nested loops in Java, in which there is a break out of the outer loop from the nested loop:

```
outerLoop:
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 85

CSC413: Organization of Programming Languages

```
for (row = 0; row < numRows; row++)
for (col = 0; col < numCols; col++) {
sum += mat[row][col];
if (sum > 1000.0)
break outerLoop;
```

C, C++, and Python include an unlabeled control statement, continue, that transfers control to the control mechanism of the smallest enclosing loop.

This is not an exit but rather a way to skip the rest of the loop statements on the current iteration without terminating the loop construct. For example, consider the following:

```
while (sum < 1000) {
getnext(value);
```

if (value < 0) continue;

sum += value;

Exercises

1. Rewrite the following pseudocode segment using a loop structure in the specified languages:

$k = (j + 13) / 27$

loop:

if $k > 10$ then goto out

$k = k + 1$

$i = 3 * k - 1$

goto loop

out: . . .

a. C, C++, Java, or C#

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 86

CSC413: Organization of Programming Languages

b. Python

2. Rewrite the following code segment using a multiple-selection statement in the following languages:

if $((k == 1) \parallel (k == 2))$ $j = 2 * k - 1$

if $((k == 3) \parallel (k == 5))$ $j = 3 * k + 1$

if $(k == 4)$ $j = 4 * k - 1$

if $((k == 6) \parallel (k == 7) \parallel (k == 8))$ $j = k - 2$

a. C, C++, Java, or C#

b. Python

Assume all variables are integer type.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 87

CSC413: Organization of Programming Languages

9. DESCRIBING LANGUAGE SYNTAX & SEMANTICS

Programming language implementors obviously must be able to determine how the expressions, statements, and program units of a language are formed, and also their intended effect when executed.

The study of programming languages, like the study of natural languages, can be divided into examinations of syntax and semantics.

- o The syntax of a programming language is the form of its expressions, statements, and program units.

- o Its semantics is the meaning of those expressions, statements, and program units.

For example, the syntax of a Java while statement is

while (boolean_expr) statement

The semantics of this statement form is that when the current value of the Boolean expression is true, the embedded statement is executed. Then control implicitly returns to the Boolean expression to repeat the process. If the Boolean expression is false, control transfers to the statement following the while construct.

A language, whether natural (such as English) or artificial (such as Java), is a set of strings of characters from some alphabet.

- The strings of a language are called sentences or statements.

- The syntax rules of a language specify which strings of characters from the language's alphabet are in the language.

English, for example, has a large and complex collection of rules for specifying the syntax of its sentences. By comparison, even the largest and most complex programming languages are syntactically very simple.

In general, languages can be formally defined in two distinct ways:

- o by recognition and
- o by generation

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 88

CSC413: Organization of Programming Languages

9.1 Language Recognizers & Generators

9.1.1 Language Recognizers

Suppose we have a language L that uses an alphabet Σ of characters. To define L formally using the recognition method,

- we would need to construct a mechanism R , called a recognition device, capable of reading strings of characters from the alphabet Σ .
- R would indicate whether a given input string was or was not in L .

If R , when fed any string of characters over Σ , accepts it only if it is in L , then R is a description of L .

Recognition devices, however, are not used to enumerate all of the sentences of a language— they have a different purpose.

The syntax analysis part of a compiler is a recognizer for the language the compiler translates.

- o In this role, the recognizer need not test all possible strings of characters from some set to determine whether each is in the language.

- Rather, it need only determine whether given programs are in the language. In effect then, the syntax analyzer determines whether the given programs are syntactically correct.

- o The structure of syntax analyzers, also known as parsers

9.1.2 Language Generators

A language generator is a device that can be used to generate the sentences of a language. We can think of the generator as having a button that produces a sentence of the language every time it is pushed.

o Because the particular sentence that is produced by a generator when its button is pushed is unpredictable, a generator seems to be a device of limited usefulness as a language descriptor.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 89

CSC413: Organization of Programming Languages

□ For example, to determine the correct syntax of a particular statement using a compiler, the programmer can only submit a speculated version and note whether the compiler accepts it.

However, people prefer certain forms of generators over recognizers because they can more easily read and understand them.

On the other hand, it is often possible to determine whether the syntax of a particular statement is correct by comparing it with the structure of the generator.

9.2 Meta-language for Describing Language Syntax

The syntax of programming languages can be described by context-free grammars. A meta-language is a language that is used to describe another language. BNF is a meta-language for programming languages. BNF is a natural notation for describing syntax. BNF uses abstractions for syntactic structures.

A simple Java assignment statement, for example, might be represented by the abstraction `<assign>` (pointed brackets are often used to delimit names of abstractions). The actual definition of `<assign>` can be given by
`<assign> → <var> = <expression>`

The text on the left side of the arrow, which is aptly called the left-hand side (LHS), is the abstraction being defined. The text to the right of the arrow is the definition of the LHS. It is called the right-Hand side (RHS) and consists of some mixture of tokens, lexemes, and references to other abstractions. Altogether, the definition is called a rule, or production.

This particular rule specifies that the abstraction `<assign>` is defined as an instance of the abstraction `<var>`, followed by the lexeme `=`, followed by an instance of the abstraction `<expression>`.

In the example rule just given, the abstractions `<var>` and `<expression>` obviously must be defined for the `<assign>` definition to be useful.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 90

CSC413: Organization of Programming Languages

NOTE:

o The lexemes of a programming language include its numeric literals, operators, and special words, among others

o Each lexeme group is represented by a name, or token.

□ So, a token of a language is a category of its lexemes.

Consider the following Java statement:

```
index = 2 * count + 17;
```

The lexemes and tokens of this statement are

Lexemes Tokens

index identifier

= equal_sign

2 int_literal

* mult_op count identifier

+ plus_op

17 int_literal

; semicolon

The abstractions in a BNF description, or grammar, are often called nonterminal symbols, or simply nonterminals, and the lexemes and tokens of the rules are called terminal symbols, or simply terminals.

A BNF description, or grammar, is a collection of rules.

o Nonterminal symbols can have two or more distinct definitions, representing two or more possible syntactic forms in the language.

Multiple definitions can be written as a single rule, with the different definitions separated by the symbol |, meaning logical OR. For example, a Java if statement can be described with the rules

```
<if_stmt> → if ( <logic_expr> ) <stmt>
```

```
<if_stmt> → if ( <logic_expr> ) <stmt> else <stmt>
```

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 91

CSC413: Organization of Programming Languages

or with the rule

```
<if_stmt> → if ( <logic_expr> ) <stmt>
```

```
| if ( <logic_expr> ) <stmt> else <stmt>
```

In these rules, <stmt> represents either a single statement or a compound statement.

Advantages of BNF

□ Using BNF, as opposed to using some informal syntax description, has at least three compelling advantages. First, BNF descriptions of the syntax of programs are clear and concise, both for humans and for software systems that use them.

□ Second, the BNF description can be used as the direct basis for the syntax analyzer.

□ Third, implementations based on BNF are relatively easy to maintain because of their modularity.

□ Grammars and Derivations

A grammar is a generative device for defining languages. The sentences of the language are generated through a sequence of applications of the rules,

o beginning with a special non-terminal of the grammar called the start symbol.

This sequence of rule applications is called a derivation. In a grammar for a complete programming language, the start symbol represents a complete program and is often named `<program>`.

The following simple grammar of a small language is used to illustrate derivations.

`<program> → begin <stmt_list> end`

`<stmt_list> → <stmt>`

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 92

CSC413: Organization of Programming Languages

`| <stmt> ; <stmt_list>`

`<stmt> → <var> = <expression>`

`<var> → A | B | C`

`<expression> → <var> + <var>`

`| <var> - <var>`

`| <var>`

The language described by the grammar above has only one statement form: assignment. A program consists of the special word `begin`, followed by a list of statements separated by semicolons, followed by the special word `end`. An expression is either a single variable or two variables separated by either a `+` or `-` operator. The only variable names in this language are `A`, `B`, and `C`.

A derivation of a program in this language follows:

`<program> ⇒ begin <stmt_list> end`

`⇒ begin <stmt>; <stmt_list> end`

`⇒ begin <var> = <expression>; <stmt_list> end`

`⇒ begin A = <expression>; <stmt_list> end`

`⇒ begin A = <var> + <var>; <stmt_list> end`

`⇒ begin A = B + <var>; <stmt_list> end`

`⇒ begin A = B + C; <stmt_list> end`

`⇒ begin A = B +; <stmt> end`

`⇒ begin A = B + C; <var> = <expression> end`

`⇒ begin A = B + C; B = <expression> end`

`⇒ begin A = B + C; B = <var> end`

`⇒ begin A = B + C; B = C end`

The derivation begins with the start symbol, in this case `<program>`. The symbol `⇒` is read “derives.”

Each successive string in the sequence is derived from the previous string by replacing one of the non-terminals with one of that non-terminal’s definitions.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 93

CSC413: Organization of Programming Languages

o Each of the strings in the derivation, including `<program>`, is called a sentential form.

In this derivation, the replaced non-terminal is always the leftmost non-terminal in the previous sentential form.

o Derivations that use this order of replacement are called leftmost derivations.

o The derivation continues until the sentential form contains no non-terminals.

o That sentential form, consisting of only terminals, or lexemes, is the generated sentence.

Consider a Grammar for simple Assignment statement

`<assign> → <id> = <expr>`

`<id> → A | B | C`

`<expr> → <id> + <expr>`

`| <id> * <expr>`

`| (<expr>)`

`| <id>`

The grammar describes assignment statements whose right sides are arithmetic expressions with multiplication and addition operators and parentheses.

For example, the statement

`A = B * (A + C)`

is generated by the leftmost derivation:

`<assign> ⇒ <id> = <expr>`

`⇒ A = <expr>`

`⇒ A = <id> * <expr>`

`⇒ A = B * <expr>`

`⇒ A = B * (<expr>)`

`⇒ A = B * (<id> + <expr>)`

`⇒ A = B * (A + <expr>)`

`⇒ A = B * (A + <id>)`

`⇒ A = B * (A + C)`

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 94

CSC413: Organization of Programming Languages

□ Parse Trees

One of the most attractive features of grammars is that they naturally describe the hierarchical syntactic structure of the sentences of the languages they define.

o These hierarchical structures are called parse trees.

For example, the parse tree of the above grammar that shows the structure of the assignment statement is as shown below.

□ Every internal node of a parse tree is labeled with a nonterminal symbol;

□ every leaf is labeled with a terminal symbol.

□ Every subtree of a parse tree describes one instance of an abstraction in the sentence.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 95

CSC413: Organization of Programming Languages

Exercises

1. The two mathematical models of language description are generation and recognition. Describe how each can define the syntax of a programming language.
2. Write a BNF description of the Boolean expressions of Java, including the three operators &&, ||, and ! and the relational expressions.
3. Consider the following grammar:

$\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b$

$\langle A \rangle \rightarrow \langle A \rangle b \mid b$

$\langle B \rangle \rightarrow b$

Which of the following sentences are in the language generated by this grammar?

- a. babb
- b. bbbabb
- c. bbaaaaabc
- d. aaaaaa

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 96

CSC413: Organization of Programming Languages

10. LEXICAL ANALYSIS & PARSING

Nearly all compilers separate the task of analyzing syntax into two distinct parts, lexical analysis and syntax analysis, although this terminology is confusing. The lexical analyzer deals with small-scale language constructs, such as names and numeric literals. The syntax analyzer deals with the large-scale constructs, such as expressions, statements, and program units.

There are three reasons why lexical analysis is separated from syntax analysis:

- o **Simplicity**— Techniques for lexical analysis are less complex than those required for syntax analysis, so the lexical-analysis process can be simpler if it is separate. Also, removing the low-level details of lexical analysis from the syntax analyzer makes the syntax analyzer both smaller and less complex.
- o **Efficiency**— Although it pays to optimize the lexical analyzer, because lexical analysis requires a significant portion of total compilation time, it is not fruitful to optimize the syntax analyzer. Separation facilitates this selective optimization.
- o **Portability**— Because the lexical analyzer reads input program files and often includes buffering of that input, it is somewhat platform dependent. However, the syntax analyzer can be platform

independent. It is always good to isolate machine-dependent parts of any software system.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 97

CSC413: Organization of Programming Languages

10.1 Lexical Analysis

A lexical analyzer is essentially a pattern matcher.

- o A pattern matcher attempts to find a substring of a given string of characters that matches a given character pattern.

- o A lexical analyzer serves as the front end of a syntax analyzer.

Technically, lexical analysis is a part of syntax analysis. A lexical analyzer performs syntax analysis at the lowest level of program structure.

- o An input program appears to a compiler as a single string of characters.

- o The lexical analyzer collects characters into logical groupings and assigns internal codes to the groupings according to their structure.

- o These logical groupings are named lexemes, and the internal codes for categories of these groupings are named tokens.

- o Lexemes are recognized by matching the input character string against character string patterns.

Although tokens are usually represented as integer values, for the sake of readability of lexical and syntax analyzers, they are often referenced through named constants.

Consider the following example of an assignment statement:

```
result = oldsum - value / 100;
```

Following are the tokens and lexemes of this statement:

Token Lexeme

IDENT result

ASSIGN_OP =

IDENT oldsum

SUB_OP -

IDENT value

DIV_OP /

INT_LIT 100

SEMICOLON ;

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 98

CSC413: Organization of Programming Languages

Lexical analyzers extract lexemes from a given input string and produce the corresponding tokens.

The lexical-analysis process includes:

- o Skipping comments and white space outside lexemes, as they are not relevant to the meaning of the program.

- o Also, the lexical analyzer inserts lexemes for user-defined names into the symbol table, which is used by later phases of the compiler.
- o Finally, lexical analyzers detect syntactic errors in tokens, such as ill-formed floating-point literals, and report such errors to the user.

There are three approaches to building a lexical analyzer:

- Write a formal description of the token patterns of the language using a descriptive language related to regular expressions. These descriptions are used as input to a software tool that automatically generates a lexical analyzer. There are many such tools available for this. The oldest of these, named *lex*, is commonly included as part of UNIX systems.
- Design a state transition diagram that describes the token patterns of the language and write a program that implements the diagram.
- Design a state transition diagram that describes the token patterns of the language and hand-construct a table-driven implementation of the state diagram.

A state transition diagram, or just state diagram, is a directed graph.

- o The nodes of a state diagram are labeled with state names.
- o The arcs are labeled with the input characters that cause the transitions among the states.
- o An arc may also include actions the lexical analyzer must perform when the transition is taken.

State diagrams of the form used for lexical analyzers are representations of a class of mathematical machines called finite automata.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 99

CSC413: Organization of Programming Languages

- Finite automata can be designed to recognize members of a class of languages called regular languages.
- Regular grammars are generative devices for regular languages.

We now illustrate lexical-analyzer construction with a state.

- o The state diagram could simply include states and transitions for each and every token pattern.
- o However, that approach results in a very large and complex diagram, because every node in the state diagram would need a transition for every character in the character set of the language being analyzed.

We therefore consider ways to simplify the state diagram

- o Suppose we need a lexical analyzer that recognizes only arithmetic expressions, including variable names and integer literals as operands. Assume that the variable names consist of strings of uppercase letters, lowercase letters, and digits but must begin with a letter. Names have no length limitation. The first thing to observe is that there are 52 different

characters (any uppercase or lowercase letter) that can begin a name, which would require 52 transitions from the transition diagram's initial state. However, a lexical analyzer is interested only in determining that it is a name and is not concerned with which specific name it happens to be. Therefore, we define a character class named LETTER for all 52 letters and use a single transition on the first letter of any name.

o Another opportunity for simplifying the transition diagram is with the integer literal tokens. There are 10 different characters that could begin an integer literal lexeme. This would require 10 transitions from the start state of the state diagram. Because specific digits are not a concern of the lexical analyzer, we can build a much more compact state diagram if we define a character class named DIGIT for digits and use a single transition on any character in this character class to a state that collects integer literals.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 100

CSC413: Organization of Programming Languages

Next, we define some utility subprograms for the common tasks inside the lexical analyzer.

- First, we need a subprogram, which we can name `getChar`, that has several duties.
- When called, `getChar` gets the next character of input from the input program and puts it in the global variable `nextChar`.
- `getChar` also must determine the character class of the input character and put it in the global variable `charClass`.
- The lexeme being built by the lexical analyzer, which could be implemented as a character string or an array, will be named `lexeme`.
- We implement the process of putting the character in `nextChar` into the string array `lexeme` in a subprogram named `addChar`.
- o This subprogram must be explicitly called because programs include some characters that need not be put in `lexeme`, for example the white-space characters between lexemes. In a more realistic lexical analyzer, comments also would not be placed in `lexeme`.
- When the lexical analyzer is called, it is convenient if the next character of input is the first character of the next lexeme.
- o Because of this, a function named `getNonBlank` is used to skip white space every time the analyzer is called.
- Finally, a subprogram named `lookup` is needed to compute the token code for the single-character tokens.
- o In our example, these are parentheses and the arithmetic operators. Token codes are numbers arbitrarily assigned to tokens by the compiler writer.

The state diagram of the lexical analyzer that recognizes names, parentheses, and arithmetic operators is shown below

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 101

CSC413: Organization of Programming Languages

10.2 Parsing

The part of the process of analyzing syntax that is referred to as syntax analysis is often called parsing. These two are used interchangeably.

Parsers for programming languages construct parse trees for given programs. In some cases, the parse tree is only implicitly constructed, meaning that perhaps only a traversal of the tree is generated.

Both parse trees and derivations include all of the syntactic information needed by a language processor.

There are two distinct goals of syntax analysis:

- o First, the syntax analyzer must check the input program to determine whether it is syntactically correct.

- When an error is found, the analyzer must produce a diagnostic message and recover. In this case, recovery means it

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 102

CSC413: Organization of Programming Languages

must get back to a normal state and continue its analysis of the input program.

- This step is required so that the compiler finds as many errors as possible during a single analysis of the input program. If it is not done well, error recovery may create more errors, or at least more error messages.

- o The second goal of syntax analysis is to produce a complete parse tree, or at least trace the structure of the complete parse tree, for syntactically correct input. The parse tree (or its trace) is used as the basis for translation.

Parsers are categorized according to the direction in which they build parse trees.

The two broad classes of parsers are:

- o top-down, in which the tree is built from the root downward to the leaves, and

- o bottom-up, in which the parse tree is built from the leaves upward to the root.

Consider the set of notational conventions for grammar symbols and strings used to make the discussion less cluttered. For formal languages, they are as follows:

1. Terminal symbols— lowercase letters at the beginning of the alphabet (a, b, . . .)
2. Nonterminal symbols— uppercase letters at the beginning of the alphabet

(A, B, . . .)

3. Terminals or nonterminals— uppercase letters at the end of the alphabet

(W, X, Y, Z)

4. Strings of terminals— lowercase letters at the end of the alphabet (w, x, y, z)

5. Mixed strings (terminals and/or nonterminals)—lowercase Greek letters

(a, b, d, g)

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 103

CSC413: Organization of Programming Languages

□ Top-Down Parsers

A top-down parser traces or builds a parse tree in pre-order.

o A pre-order traversal of a parse tree begins with the root. Each node is visited before its branches are followed. Branches from a particular node are followed in left-to-right order.

□ This corresponds to a leftmost derivation.

In terms of the derivation, a top-down parser can be described as follows:

o Given a sentential form that is part of a leftmost derivation, the parser's task is to find the next sentential form in that leftmost derivation.

o The general form of a left sentential form is xAa , whereby our notational conventions x is a string of terminal symbols,

o A is a nonterminal, and a is a mixed string. Because x contains only terminals, A is the leftmost nonterminal in the sentential form, so it is the one that must be expanded to get the next sentential form in a leftmost derivation.

Determining the next sentential form is a matter of choosing the correct grammar rule that has A as its LHS.

For example, if the current sentential form is xAa and the A -rules are $ASbB$, $AScBb$, and ASa , a top-Down parser must choose among these three rules to get the next sentential form, which could be $xbBa$, $xcBba$, or xaa . This is the parsing decision problem for top-down parsers.

Consider the production rule for the grammar:

$S \rightarrow aAS|a$

$A \rightarrow SbA|ba$

Show that the string $aabbbaa$ is derivable from the production.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 104

CSC413: Organization of Programming Languages

Solution:

Rules applied Sentential form

$S \rightarrow aAS \ aAS$

$A \rightarrow SbA \ aSbAS$

$S \rightarrow a \ aabAS$

$A \rightarrow ba \ aabbaS$

$S \rightarrow a \ aabbaa$

Note how we successively replaced the leftmost variable in the above derivation until all variable were replaced. This is a leftmost derivation.

The corresponding parse is as shown below

Exercises:

1. Discuss the bottom-up parsing and state how it differs from the top-down approach
2. How can you construct a lexical analyzer with a state diagram?
3. Describe briefly the three approaches to building a lexical analyzer.

Compiled by Orhionkpaiyo B. C. [December 2019] Updated: January 2023 Page 105